

Bimodels and Biorthogonality for Abstract Machines

APRIL TUNE, University of Bristol, United Kingdom
G. A. KAVVOS, University of Bristol, United Kingdom

We develop a compositional semantics for abstract machines, focusing on CK/CEK machines for call-by-push-value. Taking abstract machines as the primary operational semantics, we introduce bimodels, which give denotations to both programs and stacks, and environment bimodels, which extend the construction to closures. Each has a syntactic instance, built from the machine itself, and a set-theoretic instance that serves as a denotational semantics. Using biorthogonality, we define logical relations over these models and prove fundamental lemmata that are parametric in the choice of model and observation. Different instantiations yield canonicity, adequacy, operational extensionality, internal full abstraction, and a first-order simulation theorem relating the CK and CEK machines. All results are mechanized in Agda.

CCS Concepts: • **Theory of computation** → **Abstract machines**; **Denotational semantics**; *Program reasoning*.

Additional Key Words and Phrases: abstract machines, call-by-push-value, biorthogonality, orthogonality, logical relations, mechanized metatheory

ACM Reference Format:
April Tune and G. A. Kavvos. 2026. Bimodels and Biorthogonality for Abstract Machines. *Proc. ACM Program. Lang.* 10, ICFP, Article 288 (August 2026), 30 pages. https://doi.org/10.1145/3828686

1 Introduction

The extra machinery is avoided by the use of the rules, making the exploration of syntactical structure implicit rather than drearily explicit. Of course, abstract machines are important for the actual implementation of programming languages; indeed it would be good to have a general theory of such machines.

Gordon Plotkin, "The origins of structural operational semantics" [2004a]

Over the past 45 years much of programming language theory has been based on small-step, *structural operational semantics*, as developed and popularised by Plotkin [2004a,b]. The animating idea is that programs can be organized into a transition system whose transitions are given by a handful of *structural* rules, formulated by induction on syntax. This paradigm is attractive for studying small languages. When coupled with Wright and Felleisen's principles of progress and preservation [1994], it forms the backbone of introductory textbooks [Harper 2016; Pierce 2002].

However, small-step semantics has significant limitations. First, it is *not compositional*: the meaning of a compound program is not given through the meaning of its parts [Plotkin 2004a]. It is thus hard to answer even simple questions, e.g. equivalence of program fragments. A substantial amount of effort has gone into developing operational reasoning methods; see e.g. Mason, Smith, et al. [1996] and Pitts [2011, 1997, 2005]. Second, small-step semantics is *often far from an implementation*. For example, the operational semantics of functions is usually formulated using substitution, which

Authors' Contact Information: April Tune, University of Bristol, School of Computer Science, Bristol, United Kingdom, april.tune@bristol.ac.uk; G. A. Kavvos, University of Bristol, School of Computer Science, Bristol, United Kingdom, alex.kavvos@bristol.ac.uk.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2475-1421/2026/8-ART288
https://doi.org/10.1145/3828686

is seldom implemented in practice. Thus, analyses based on small-step semantics do not by themselves justify a particular implementation unless they are connected to it by a separate theorem; for example, CompCert includes proofs of a dozen such theorems, one for each compilation phase [Leroy 2009]. The alternative technique of *denotational semantics* is compositional, but requires an *adequacy* theorem connecting denotations to program execution [Milner 1975; Plotkin 1977].

This paper takes abstract machines as the **primary operational semantics**. Indeed, abstract machines are a tool found at the heart of many language implementations [Diehl et al. 2000; Peyton Jones 1987]. We develop a semantics that *begins* with an abstract machine, instead of deriving it from a definitional interpreter or evaluator (as in the style of Reynolds [1998], Ager et al. [2003], and Danvy [2005]). This keeps our semantics somewhat closer to implementation, while still abstracting away implementation details. On the other hand, our approach is decidedly **denotational**: it maps each moving part of the abstract machine to a suitable semantic value. This gives us the benefits of a compositional semantics, enabling access to algebraic and logical techniques.

The components of an abstract machine usually admit a division into the *program* and its surrounding context, or *milieu*. While a milieu may have many parts, we consider only the minimum, i.e. a *stack*. $\langle M \mid K \rangle$ stands for the pair of a program M and a stack K. We introduce the notion of a **bimodel**, which gives denotations to both, thereby making stacks *first-class semantic entities*.

This division between program and milieu allows us to use the technique of **biorthogonality**. That is, given a choice of a *pole* $\perp\!\!\!\perp$, i.e. a relation over (denotations of) pairs $\langle M \mid K \rangle$ that is closed under inverse evaluation, we define biorthogonal logical relations over bimodels. The pole identifies states of the machine that, when run, yield desirable *observations*. The biorthogonal logical relations then lift this observational criterion to relations on denotations. This setting allows us to prove the *fundamental lemma of biorthogonal logical relations* for denotations $[\![M]\!]$ of programs independently of the particular pole, adapting the highly influential memo of Plotkin [1973] to biorthogonality.

As formulated in this paper, bimodels are intensional models. Instead of being quotiented mathematical objects (e.g. domains), they come equipped with a *dynamic relation* $\rightsquigarrow$ between pairs of programs and stacks, the denotational analogue of a transition $\langle M \mid K \rangle \rightsquigarrow \langle M' \mid K' \rangle$ of the abstract machine. For example, our syntactic model does *not* require quotienting by $\beta\eta$-equality. Instead, various notions of equality arise as biorthogonal logical relations, after instantiation with the appropriate pole. This enables direct mechanization in a proof assistant, without using quotient types or setoids. We also believe it might enable a simpler account of more intensional models, e.g. game semantics [Levy 2003b, §8], which often require quotienting to form appropriate categories. Finally, note that *intensionality is not essential to the notion of bimodel*: we are free to require certain equations between pairs $\langle M \mid K \rangle$ to hold, with all the attendant issues in mechanization.

We develop a notion of bimodel and pole for two different abstract machines, a **CK machine** and a **CEK machine** [Felleisen and Friedman 1987]. We prove the unary and binary fundamental lemmata of biorthogonal logical relations for both. For each type of bimodel we give two instances: a syntactic bimodel, and a set-theoretic bimodel. Applying the fundamental lemma to various combinations of these bimodels yields several results as corollaries: *canonicity*; *adequacy* of the set-theoretic model for the abstract machine; *operational extensionality*, i.e. the coincidence of the syntactic logical relation with a closed-instantiations-of-use (CIU) relation; and a *simulation theorem* establishing that, at first-order types, every value returned by the CK machine is also returned by the CEK machine, up to the normal form of value closures. Within the same framework we also establish the *internal full abstraction* of the equality pole for the set-theoretic model: we prove that the logical relation induced by an 'equality' pole is, in fact, equality.

To make the presentation as simple as possible we base it on **Call-by-Push-Value (CBPV)**, a typed λ-calculus that separates *values* (inert data) from *computations* (processes that may perform effects) [Levy 2003b]. Even though we do not treat either recursion or effects in this work, CBPV

enables a uniform treatment of them. However, even in the absence of effects, the structure of CBPV induces canonical CK and CEK machines, giving a particularly crisp account.

Contributions. In summary, the contributions of this paper are:

- The notion of **bimodel**, which abstracts the structure of an abstract machine by giving denotations to both programs and stacks (Section 4).
- A theory of **biorthogonal logical relations over bimodels**, parametric in the choice of bimodel and pole, together with a fundamental lemma that is proved once and for all by adapting the logical relations method of Plotkin [1973] to biorthogonality (Section 4.3).
- **Applications of CK bimodels**: canonicity, adequacy, operational extensionality, and internal full abstraction, obtained by varying the bimodel and pole (Section 5).
- A **CEK machine** for CBPV, which to the best of our knowledge has not appeared before in the literature (Section 6), and an attendant notion of **environment bimodel** to handle closures and environments (Section 7). We use this abstraction to obtain canonicity, adequacy, and a simulation theorem relating the CK and CEK machines (Section 8).
- A mechanization of all our results in Agda.

2 A User's Guide to CBPV

Call-by-push-value (CBPV) is a typed λ-calculus introduced by Levy [1999] that decomposes evaluation order into fine-grained primitives. The key idea is a strict separation between values and computations: *a value is, while a computation does.* Values are inert data—variables, pairs, sums—while computations pattern-match on them and may have effects. We give a brief introduction; for details see Levy [2003b, 1999, 2006].

The types and typing rules are presented in Fig. 1. We consider value types A consisting of a unit type $\mathbf{1}$, binary sums $A_1 + A_2$, binary products $A_1 \times A_2$, and thunks $U\underline{C}$. A thunk is a suspended computation: a value that, when forced, reveals a computation of type $\underline{C}$. Computation types $\underline{C}$ comprise returner types FA and functions $A \to \underline{C}$. In full CBPV these classify computations that may perform effects, but in this paper we study the pure fragment. The computation types of CBPV may also be thought of as the types of stacks [Jiang et al. 2025].

The terms of CBPV are divided into values and computations. Values are built from variables x, the unit value $\langle\rangle$, pairs (V_1, V_2), sum injections $\mathsf{inl}(V)$ and $\mathsf{inr}(V)$, and thunks $\mathsf{thunk}\ M$. Note that a closed value is necessarily *canonical*, i.e. headed by a constructor.

The corresponding eliminators appear as computations: $\mathsf{split}(V; x_1, x_2.\, M)$ destructs a pair, $\mathsf{case}(V; x_1.\, M_1; x_2.\, M_2)$ performs case analysis on a sum, and $\mathsf{force}\ V$ forces a thunk. The remaining computation forms are λ-abstraction $\lambda x.\, M$, application $M(V)$, the returner $\mathsf{return}\ V$, and sequencing $M\ \mathsf{to}\ x.\, N$, which binds the value returned by M to the variable x in N.

We also single out a subset of value types called *first-order types*, ranged over by σ. These are value types that *do not include thunks.* First-order types are important because their closed values have fully observable structure, and can be 'serialised' uniquely. This makes their mathematical semantics simple; for example, they can be modelled by discrete domains [Kavvos 2025].

We write CompTy for the set of computation types $\underline{C}$; it indexes the logical relations and poles of later sections. We also write $\mathsf{Val}(A) \stackrel{\text{def}}{=} \{V \mid \vdash^{\mathrm{v}} V : A\}$ and $\mathsf{Comp}(\underline{C}) \stackrel{\text{def}}{=} \{M \mid \vdash^{\mathrm{c}} M : \underline{C}\}$ for the sets of closed values and computations, respectively.

3 The CK Machine

We now show how to execute programs of CBPV by presenting an abstract machine—in fact a CK machine, as introduced by Felleisen and Friedman [1987] and adapted to CBPV by Levy [2003a]; see also Levy [2003b, §2.3]. The components of the machine are defined in Fig. 2.

$\boxed{A, B}$ **Value types**

$$A, B ::= \mathbf{1} \mid A_1 + A_2 \mid A_1 \times A_2 \mid U\underline{C}$$

(unit; sums; products; thunks)

$\boxed{\sigma}$ **First-order types**

$$\sigma ::= \mathbf{1} \mid \sigma_1 + \sigma_2 \mid \sigma_1 \times \sigma_2$$

$\boxed{\underline{C}, \underline{D}}$ **Computation types**

$$\underline{C}, \underline{D} ::= FA \mid A \to \underline{C}$$

(returners; functions)

$\boxed{\Gamma}$ **Contexts**

$$\Gamma ::= \cdot \mid \Gamma, x : A$$

(empty; extension)

$\boxed{\Gamma \vdash^{\mathrm{v}} V : A}$ **Values**

$\boxed{\Gamma \vdash^{\mathrm{c}} M : \underline{C}}$ **Computations**

$$\textsc{Var}\ \frac{}{\Gamma, x : A, \Gamma' \vdash^{\mathrm{v}} x : A} \qquad \textsc{Unit}\ \frac{}{\Gamma \vdash^{\mathrm{v}} \langle\rangle : \mathbf{1}}$$

$$\textsc{Pair}\ \frac{\Gamma \vdash^{\mathrm{v}} V_1 : A_1 \qquad \Gamma \vdash^{\mathrm{v}} V_2 : A_2}{\Gamma \vdash^{\mathrm{v}} (V_1, V_2) : A_1 \times A_2} \qquad \textsc{Inl}\ \frac{\Gamma \vdash^{\mathrm{v}} V : A_1}{\Gamma \vdash^{\mathrm{v}} \mathsf{inl}(V) : A_1 + A_2}$$

$$\textsc{Inr}\ \frac{\Gamma \vdash^{\mathrm{v}} V : A_2}{\Gamma \vdash^{\mathrm{v}} \mathsf{inr}(V) : A_1 + A_2} \qquad \textsc{Thunk}\ \frac{\Gamma \vdash^{\mathrm{c}} M : \underline{C}}{\Gamma \vdash^{\mathrm{v}} \mathsf{thunk}\ M : U\underline{C}} \qquad \textsc{Lam}\ \frac{\Gamma, x : A \vdash^{\mathrm{c}} M : \underline{C}}{\Gamma \vdash^{\mathrm{c}} \lambda x.\, M : A \to \underline{C}}$$

$$\textsc{App}\ \frac{\Gamma \vdash^{\mathrm{c}} M : A \to \underline{C} \qquad \Gamma \vdash^{\mathrm{v}} V : A}{\Gamma \vdash^{\mathrm{c}} M(V) : \underline{C}} \qquad \textsc{Return}\ \frac{\Gamma \vdash^{\mathrm{v}} V : A}{\Gamma \vdash^{\mathrm{c}} \mathsf{return}\ V : FA} \qquad \textsc{Bind}\ \frac{\Gamma \vdash^{\mathrm{c}} M : FA \qquad \Gamma, x : A \vdash^{\mathrm{c}} N : \underline{C}}{\Gamma \vdash^{\mathrm{c}} M\ \mathsf{to}\ x.\, N : \underline{C}}$$

$$\textsc{Force}\ \frac{\Gamma \vdash^{\mathrm{v}} V : U\underline{C}}{\Gamma \vdash^{\mathrm{c}} \mathsf{force}\ V : \underline{C}} \qquad \textsc{Split}\ \frac{\Gamma \vdash^{\mathrm{v}} V : A_1 \times A_2 \qquad \Gamma, x_1 : A_1, x_2 : A_2 \vdash^{\mathrm{c}} M : \underline{C}}{\Gamma \vdash^{\mathrm{c}} \mathsf{split}(V; x_1, x_2.\, M) : \underline{C}}$$

$$\textsc{Case}\ \frac{\Gamma \vdash^{\mathrm{v}} V : A_1 + A_2 \qquad \Gamma, x_1 : A_1 \vdash^{\mathrm{c}} M_1 : \underline{C} \qquad \Gamma, x_2 : A_2 \vdash^{\mathrm{c}} M_2 : \underline{C}}{\Gamma \vdash^{\mathrm{c}} \mathsf{case}(V; x_1.\, M_1; x_2.\, M_2) : \underline{C}}$$

Fig. 1. Call-by-Push-Value: types and typing rules (`ValTy`, `CompTy`, `FirstOrder`, `Context`, $_ \vdash^{\mathrm{v}} _$, $_ \vdash^{\mathrm{c}} _$).

The state of a CK machine is a pair $\langle M \mid K \rangle$ consisting of a *computation* M and a *stack* K.[1] The computation is the program currently being executed, and the stack records what remains to be done once the current computation finishes. The objective of a machine is to turn a computation into a *terminal computation*, i.e. either a returner $\mathsf{return}\ V$ or a λ-abstraction $\lambda x.\, M$. These comprise the two possible outcomes of running a computation: it either produces a value or it awaits an argument. We write $\mathsf{Tmn}(\underline{C}) \stackrel{\text{def}}{=} \{T \mid \vdash^{\mathrm{c}} T : \underline{C}\ \text{terminal}\}$ for the set of terminal computations at $\underline{C}$.

A stack is either the empty stack $\odot$, a stack $V :: K$ with a value V at its top (recording a pending application), or a stack $\mathsf{to}\ x.\, N :: K$ with a *continuation* at its top (recording the next computation). A closed machine proceeds by pattern-matching on the current computation. The elimination forms of computation types push onto the stack: an application $M(V)$ pushes the argument V onto

[1]The initialism CK stands for Computation and stacK, or—originally—Kontinuation.

$\boxed{K}$ **Stacks**

$$K ::= \odot \mid V :: K \mid \mathsf{to}\, x.\, N :: K$$

$\boxed{T}$ **Terminal computations**

$$T ::= \mathsf{return}\, V \mid \lambda x.\, M$$

$\boxed{S}$ **Machines**

$$S ::= \langle M \mid K \rangle$$

$\boxed{S_T}$ **Terminal machines**

$$S_T ::= \langle T \mid \odot \rangle$$

$\boxed{S \rightsquigarrow S'}$ **Transition rules**

$$\langle M(V) \mid K \rangle \rightsquigarrow \langle M \mid V :: K \rangle \qquad \langle M\ \mathsf{to}\ x.\, N \mid K \rangle \rightsquigarrow \langle M \mid \mathsf{to}\, x.\, N :: K \rangle$$
$$\langle \lambda x.\, M \mid V :: K \rangle \rightsquigarrow \langle M[V/x] \mid K \rangle \qquad \langle \mathsf{return}\ V \mid \mathsf{to}\, x.\, N :: K \rangle \rightsquigarrow \langle N[V/x] \mid K \rangle$$

$$\langle \mathsf{force}(\mathsf{thunk}\ M) \mid K \rangle \rightsquigarrow \langle M \mid K \rangle$$
$$\langle \mathsf{split}((V_1, V_2); x_1, x_2.\, M) \mid K \rangle \rightsquigarrow \langle M[V_1, V_2/x_1, x_2] \mid K \rangle$$
$$\langle \mathsf{case}(\mathsf{inl}(V); x_1.\, M_1; x_2.\, M_2) \mid K \rangle \rightsquigarrow \langle M_1[V/x_1] \mid K \rangle$$
$$\langle \mathsf{case}(\mathsf{inr}(V); x_1.\, M_1; x_2.\, M_2) \mid K \rangle \rightsquigarrow \langle M_2[V/x_2] \mid K \rangle$$

$\boxed{\Gamma \vdash K : \underline{C} \multimap \underline{D}}$ **Stack types**

Stk-Empty
$$\frac{}{\Gamma \vdash \odot : \underline{C} \multimap \underline{C}}$$

Stk-Val
$$\frac{\Gamma \vdash^{\mathrm{v}} V : A \qquad \Gamma \vdash K : \underline{C} \multimap \underline{D}}{\Gamma \vdash V :: K : (A \to \underline{C}) \multimap \underline{D}}$$

Stk-Cont
$$\frac{\Gamma, x : A \vdash^{\mathrm{c}} N : \underline{C} \qquad \Gamma \vdash K : \underline{C} \multimap \underline{D}}{\Gamma \vdash \mathsf{to}\, x.\, N :: K : FA \multimap \underline{D}}$$

$\boxed{\Gamma \vdash \langle M \mid K \rangle : \underline{D}}$ **Machine types**

Machine
$$\frac{\Gamma \vdash^{\mathrm{c}} M : \underline{C} \qquad \Gamma \vdash K : \underline{C} \multimap \underline{D}}{\Gamma \vdash \langle M \mid K \rangle : \underline{D}}$$

Fig. 2. The CK machine for CBPV ($_ \rightsquigarrow _$, $_ \vdash _ \multimap _$).

the stack, and continues evaluating M to a λ-abstraction. Similarly, sequencing M to $x.\, N$ pushes the continuation $x.\, N$ onto the stack, and continues evaluating M to a returner.

When the current computation reaches a terminal form it interacts with the top of the stack. A λ-abstraction meeting a value on the stack performs a substitution: $\langle \lambda x.\, M \mid V :: K \rangle$ steps to $\langle M[V/x] \mid K \rangle$. A return V meeting a continuation installs the continuation with the value V substituted for the bound variable: $\langle \mathsf{return}\ V \mid \mathsf{to}\, x.\, N :: K \rangle$ steps to $\langle N[V/x] \mid K \rangle$.

The value eliminators rely on the implicit invariants of CBPV. Recall that the machine operates on closed terms, so the eliminated value is headed by a constructor. As a result, the machine can always make progress: a $\mathsf{force}(\mathsf{thunk}\ M)$ unfolds the thunk, a $\mathsf{split}((V_1, V_2); x_1, x_2.\, M)$ destructs the pair and substitutes its components for x_1 and x_2, and a case analysis on $\mathsf{inl}(V)$ or $\mathsf{inr}(V)$ selects the appropriate branch and substitutes V for the bound variable.

$$\mathcal{R}^v_{\mathbf{1}}\ ()\ \stackrel{\text{def}}{\equiv}\ \top \qquad \mathcal{R}^v_{A_1+A_2}\ \mathsf{inl}(V)\ \stackrel{\text{def}}{\equiv}\ \mathcal{R}^v_{A_1}\ V$$

$$\mathcal{R}^v_{A_1+A_2}\ \mathsf{inr}(V)\ \stackrel{\text{def}}{\equiv}\ \mathcal{R}^v_{A_2}\ V \qquad \mathcal{R}^v_{A_1\times A_2}\ (V_1,V_2)\ \stackrel{\text{def}}{\equiv}\ \mathcal{R}^v_{A_1}\ V_1 \wedge \mathcal{R}^v_{A_2}\ V_2$$

$$\mathcal{R}^v_{U\underline{C}}\ \mathsf{thunk}\ M\ \stackrel{\text{def}}{\equiv} \mathcal{R}^c_{\underline{C}}\ M$$

$$\mathcal{R}^t_{FA}\ \mathsf{return}\ V\ \stackrel{\text{def}}{\equiv} \mathcal{R}^v_{A}\ V$$

$$\mathcal{R}^t_{A\to\underline{C}}\ \lambda x.M\ \stackrel{\text{def}}{\equiv} \forall V.\ \mathcal{R}^v_A\ V \Rightarrow \mathcal{R}^c_{\underline{C}}\ M[V/x]$$

$$\mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K\ \stackrel{\text{def}}{\equiv} \forall T.\ \mathcal{R}^t_{\underline{C}}\ T \Rightarrow \langle T \mid K\rangle \in \perp\!\!\!\perp_{\underline{D}}$$

$$\mathcal{R}^c_{\underline{C}}\ M\ \stackrel{\text{def}}{\equiv} \forall \underline{D}, K.\ \mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K \Rightarrow \langle M \mid K\rangle \in \perp\!\!\!\perp_{\underline{D}}$$

Fig. 3. A syntactic logical relation for canonicity in the CK machine ($\mathsf{R}^{\mathsf{v}}_\ni_$, $\mathsf{R}^{\mathsf{t}}_\ni_,_$, $\mathsf{R}^{\mathsf{k}}_\multimap_\ni_$, $\mathsf{R}^{\mathsf{c}}_\ni_$).

We can also type stacks, and hence machines [Levy 2003a]. The judgement $\Gamma \vdash K : \underline{C} \multimap \underline{D}$ means that K is a stack that, when *dismantled* onto a computation of type $\underline{C}$, yields a computation of type $\underline{D}$. Likewise, the judgement $\Gamma \vdash \langle M \mid K\rangle : \underline{D}$ means that the machine state $\langle M \mid K\rangle$ is well-typed, in the sense that dismantling K onto M yields an overall computation of type $\underline{D}$. We write $\mathsf{Stk}(\underline{C},\underline{D}) \stackrel{\text{def}}{=} \{K \mid \vdash K : \underline{C} \multimap \underline{D}\}$ for the set of stacks of the respective types. Note that stacks may have free variables; this is necessary for a uniform, algebraic notion of bimodel (Section 4).

Canonicity. Recall that the objective of the machine is to reach a terminal state $\langle T \mid \odot\rangle$ where T is terminal and the stack is empty. In fact, we can prove that CBPV without effects or recursion satisfies *canonicity*, i.e. every closed computation terminates this way when loaded in the CK machine with the empty stack. In the rest of this section we sketch this proof to illustrate biorthogonal logical relations at the syntactic level (`CK.SyntacticLR.PureCanonicity`).

The first step to a biorthogonality proof is to define a *pole*, i.e. a predicate carving out machines with desirable properties at every computation type. As we wish to prove canonicity, we take the pole to be the set of well-typed, closed machines that terminate (`Terminates`):

$$\perp\!\!\!\perp_{\underline{C}} \stackrel{\text{def}}{=} \{\vdash \langle M \mid K\rangle : \underline{C} \mid \langle M \mid K\rangle \rightsquigarrow^* \langle T \mid \odot\rangle \text{ for some terminal } T\}$$

$\perp\!\!\!\perp$ must be closed under inverse transitions, i.e. $S \rightsquigarrow S'$ with $S' \in \perp\!\!\!\perp_{\underline{C}}$ implies $S \in \perp\!\!\!\perp_{\underline{C}}$.

We define four mutually dependent predicates, which are indexed by types:

$$\mathcal{R}^v_A \subseteq \mathsf{Val}(A), \qquad \mathcal{R}^t_{\underline{C}} \subseteq \mathsf{Tmn}(\underline{C}), \qquad \mathcal{R}^k_{\underline{C}\multimap\underline{D}} \subseteq \mathsf{Stk}(\underline{C},\underline{D}), \qquad \mathcal{R}^c_{\underline{C}} \subseteq \mathsf{Comp}(\underline{C}).$$

These hold on *closed* values, terminal computations, stacks, and computations respectively, all indexed by their type. The defining clauses are given in Fig. 3.

The *value relation* is defined by induction on the value type, followed by induction on the structure of the value. Recall that closed values cannot be variables, so they must be headed by a constructor. The relation pattern-matches on this constructor, and requires that the constituent values are also in the relation. For thunks it defers to the computation relation.

The *terminal computation relation* is defined by induction on the computation type, followed by induction on the structure of the terminal computation (which is determined by its type). The relation then asks that any constituent value is in the value relation, and that all ways of eliminating the terminal computation (potentially using values in the relation) also remain in the relation.

The stack and computation relations are defined by *orthogonality* with respect to the pole. The *stack relation* contains a stack just if coupling it with all 'good' terminals (i.e. in the relation) yields a machine in the pole. The *computation relation* contains a computation just if coupling it with all 'good' stacks (i.e. in the relation) yields a machine in the pole.

These definitions are not circular: they are well-founded because types get smaller. For $\mathcal{R}^k_{\underline{C} \multimap \underline{D}}$ we consider only the first type parameter $\underline{C}$ as a metric. The value relation depends on itself and the computation relation, but only at smaller types. The terminal computation relation depends on both the value and the computation relation, but again at smaller types. The computation relation uses only the stack relation, which in turn uses only the terminal computation relation. Hence, both go through the terminal computation relation, which makes types smaller.

We can now show that

Lemma 3.1 (Stack constructors preserve the relation, ::-good, to::-good).
(1) *If* $\mathcal{R}^k_{\underline{C} \multimap \underline{D}}\ K$ *and* $\mathcal{R}^v_A\ V$ *then* $\mathcal{R}^k_{(A \to \underline{C}) \multimap \underline{D}}\ V :: K$.
(2) *If* $\mathcal{R}^k_{\underline{C} \multimap \underline{D}}\ K$ *and for all* $\mathcal{R}^v_A\ V$ *we have* $\mathcal{R}^c_{\underline{C}}\ N[V/x]$, *then* $\mathcal{R}^k_{FA \multimap \underline{D}}\ to\ x.\, N :: K$.

Proof. We only prove (1). Given $\mathcal{R}^t_{A \to \underline{C}}\ \lambda x.\, M$ the assumptions give us $\langle M[V/x] \mid K \rangle \in \perp\!\!\!\perp_{\underline{D}}$. But the machine takes a step $\langle \lambda x.\, M \mid V :: K \rangle \rightsquigarrow \langle M[V/x] \mid K \rangle$, hence $\langle \lambda x.\, M \mid V :: K \rangle \in \perp\!\!\!\perp_{\underline{D}}$. □

We define a *closing substitution* $\gamma : \Gamma$ to be a map from every variable $x : A$ in Γ to a closed value $\vdash^v V : A$, and write $V[\gamma]$ and $M[\gamma]$ for terms that arise from applying γ as a simultaneous substitution. We write $\mathcal{R}^\Gamma\ \gamma$ to mean that $\mathcal{R}^v_A\ \gamma(x)$ for each $x : A \in \Gamma$. We can now show the

Lemma 3.2 (Fundamental Lemma for Canonicity, rel-val, rel-com).
(1) *If* $\Gamma \vdash^v V : A$ *and* $\mathcal{R}^\Gamma\ \gamma$ *then* $\mathcal{R}^v_A\ V[\gamma]$.
(2) *If* $\Gamma \vdash^c M : \underline{C}$ *and* $\mathcal{R}^\Gamma\ \gamma$ *then* $\mathcal{R}^c_{\underline{C}}\ M[\gamma]$.

Proof. By induction on typing derivations, using backward closure of the pole. We show App as an example. Given $\Gamma \vdash^c M(V) : \underline{C}$, we know $\mathcal{R}^c_{A \to \underline{C}}\ M[\gamma]$ and $\mathcal{R}^v_A\ V[\gamma]$ by induction. For any $\mathcal{R}^k_{\underline{C} \multimap \underline{D}}\ K$, Lemma 3.1(1) gives $\mathcal{R}^k_{(A \to \underline{C}) \multimap \underline{D}}\ V[\gamma] :: K$. Therefore, since $\mathcal{R}^c_{A \to \underline{C}}\ M[\gamma]$, we have $\langle M[\gamma] \mid V[\gamma] :: K \rangle \in \perp\!\!\!\perp_{\underline{D}}$. But by backward closure of the pole, $\langle M[\gamma](V[\gamma]) \mid K \rangle \in \perp\!\!\!\perp_{\underline{D}}$. □

Notice that, unlike many similar arguments, *we need not prove a fundamental lemma for stacks.*[2] We only need the empty stack to be in the relation:

Lemma 3.3 (nil-good). *For every computation type* $\underline{C}$ *we have* $\mathcal{R}^k_{\underline{C} \multimap \underline{C}}\ \odot$.

This holds because for every terminal T of type $\underline{C}$ we have $\langle T \mid \odot \rangle \in \perp\!\!\!\perp_{\underline{C}}$. We can then prove the

Theorem 3.4 (canonicity). *If* $\vdash^c M : \underline{C}$ *then* $\langle M \mid \odot \rangle \rightsquigarrow^* \langle T \mid \odot \rangle$ *for some terminal* T.

Proof. By Lemma 3.2, $\mathcal{R}^c_{\underline{C}}\ M$. Since $\mathcal{R}^k_{\underline{C} \multimap \underline{C}}\ \odot$, we get $\langle M \mid \odot \rangle \in \perp\!\!\!\perp_{\underline{C}}$. □

4 Bimodels

A bimodel is a denotational model of CBPV that follows the algebraic structure of its CK machine. We present bimodels in the style of 'type theory in type theory' [Altenkirch and Kaposi 2016], i.e. as a record of sorts, families, operations, and equations. This makes formalisation in a dependently typed proof assistant transparent. However, nothing depends on this choice of metatheory; the same structure works in set theory just as well.

[2]Nevertheless, such a lemma holds whenever $\mathcal{R}^k_{\underline{C} \multimap \underline{C}}\ \odot$, using induction and Lemma 3.1.

Sorts: $\mathsf{Ctx} : \mathsf{Set}_1$, $\mathsf{VTy} : \mathsf{Set}_1$, $\mathsf{CTy} : \mathsf{Set}_1$

Families:

$$\begin{aligned}
\mathsf{Val}\ \Gamma\ A &: \mathsf{Set} \\
\mathsf{Com}\ \Gamma\ \underline{C} &: \mathsf{Set} \\
\mathsf{Stk}\ \Gamma\ \underline{C}\ \underline{D} &: \mathsf{Set} \\
\mathsf{Sub}\ \Delta\ \Gamma &: \mathsf{Set}
\end{aligned}$$

Type and context operations:

$$\begin{aligned}
\varnothing &: \mathsf{Ctx} \\
- \mathbin{.} - &: \mathsf{Ctx} \to \mathsf{VTy} \to \mathsf{Ctx} \\
\mathbf{1} &: \mathsf{VTy} \\
- + -,\ - \times - &: \mathsf{VTy} \to \mathsf{VTy} \to \mathsf{VTy} \\
U &: \mathsf{CTy} \to \mathsf{VTy} \\
F &: \mathsf{VTy} \to \mathsf{CTy} \\
- \to - &: \mathsf{VTy} \to \mathsf{CTy} \to \mathsf{CTy}
\end{aligned}$$

Substitution:

$$\begin{aligned}
\epsilon &: \mathsf{Sub}\ \Gamma\ \varnothing \\
\mathsf{id} &: \mathsf{Sub}\ \Gamma\ \Gamma \\
\mathsf{p} &: \mathsf{Sub}\ \Gamma\mathbin{.}A\ \Gamma \\
\mathsf{q} &: \mathsf{Val}\ \Gamma\mathbin{.}A\ A \\
- \cdot - &: \mathsf{Sub}\ \Delta\ \Gamma \to \mathsf{Val}\ \Delta\ A \to \mathsf{Sub}\ \Delta\ (\Gamma\mathbin{.}A) \\
- \circ - &: \mathsf{Sub}\ \Delta\ \Gamma \to \mathsf{Sub}\ \Theta\ \Delta \to \mathsf{Sub}\ \Theta\ \Gamma \\
-[-]^{\mathsf{v}} &: \mathsf{Val}\ \Gamma\ A \to \mathsf{Sub}\ \Delta\ \Gamma \to \mathsf{Val}\ \Delta\ A \\
-[-]^{\mathsf{c}} &: \mathsf{Com}\ \Gamma\ \underline{C} \to \mathsf{Sub}\ \Delta\ \Gamma \to \mathsf{Com}\ \Delta\ \underline{C} \\
-[-]^{\mathsf{s}} &: \mathsf{Stk}\ \Gamma\ \underline{C}\ \underline{D} \to \mathsf{Sub}\ \Delta\ \Gamma \to \mathsf{Stk}\ \Delta\ \underline{C}\ \underline{D}
\end{aligned}$$

subject to the standard laws, e.g.

$$\mathsf{q}[\sigma \cdot V]^{\mathsf{v}} = V \qquad \mathsf{p} \circ (\sigma \cdot V) = \sigma.$$

Resolution functions:

$$\begin{aligned}
&\mathsf{resolve}_+(-) : \mathsf{Val}\ \varnothing\ (A_1 + A_2) \to \mathsf{Val}\ \varnothing\ A_1 + \mathsf{Val}\ \varnothing\ A_2 \\
&\mathsf{resolve}_\times(-) : \mathsf{Val}\ \varnothing\ (A_1 \times A_2) \to \mathsf{Val}\ \varnothing\ A_1 \times \mathsf{Val}\ \varnothing\ A_2
\end{aligned}$$

Term constructors: For each typing rule in Fig. 1, a corresponding operation. For example,

$$\begin{aligned}
\mathsf{return} &: (\Gamma : \mathsf{Ctx}) \to (A : \mathsf{VTy}) \to \mathsf{Val}\ \Gamma\ A \to \mathsf{Com}\ \Gamma\ (FA) \\
\mathsf{to} &: (\Gamma : \mathsf{Ctx}) \to (A : \mathsf{VTy}) \to (C : \mathsf{CTy}) \to \mathsf{Com}\ \Gamma\ (FA) \to \mathsf{Com}\ \Gamma\mathbin{.}A\ \underline{C} \to \mathsf{Com}\ \Gamma\ \underline{C}
\end{aligned}$$

Constructor coherence: Resolution functions agree with constructors, e.g.

$$\mathsf{resolve}_+(\mathsf{inl}(V)) = \mathsf{inj}_1(V), \quad \mathsf{resolve}_+(\mathsf{inr}(V)) = \mathsf{inj}_2(V), \quad \mathsf{resolve}_\times(\mathsf{pair}(V_1, V_2)) = (V_1, V_2)$$

where $\mathsf{inj}_i : \mathsf{Val}\ \varnothing\ A_i \to \mathsf{Val}\ \varnothing\ A_1 + \mathsf{Val}\ \varnothing\ A_2$ are the injections in the metatheory.

Stack constructors: For each stack typing rule in Fig. 2, a corresponding operation.

Machine states and transition: A relation

$$(- \rightsquigarrow -) : \mathsf{State}\ \underline{C} \to \mathsf{State}\ \underline{C} \to \mathsf{Set}$$

where $\mathsf{State}\ \underline{C} \stackrel{\text{def}}{=} (\underline{D} : \mathsf{CTy}) \times (\mathsf{Com}\ \varnothing\ \underline{D} \times \mathsf{Stk}\ \varnothing\ \underline{D}\ \underline{C})$, closed under the rules of Fig. 2 in terms of the bimodel operations, e.g. $(M(V), K) \rightsquigarrow (M, V :: K)$.

Fig. 4. A bimodel (`Bimodel`).

The full theory of a bimodel $\mathfrak{B}$ is given in Fig. 4. The sorts Ctx, VTy, and CTy correspond to contexts, value types, and computation types. These live in a larger universe (Set_1) in order to accommodate models whose types come from a smaller universe. For each syntactic object of CBPV there are corresponding sorts and families: the sorts above for contexts, value types, and computation types; and families $\mathsf{Val}\ \Gamma\ A$ for values $\Gamma \vdash^{\mathsf{v}} V : A$, $\mathsf{Com}\ \Gamma\ \underline{C}$ for computations $\Gamma \vdash^{\mathsf{c}} M : \underline{C}$, $\mathsf{Stk}\ \Gamma\ \underline{C}\ \underline{D}$ for stacks $\Gamma \vdash K : \underline{C} \multimap \underline{D}$, and $\mathsf{Sub}\ \Delta\ \Gamma$ for substitutions. The last family is the denotational analogue of a map $\gamma : \Delta \to \Gamma$ between two contexts, which maps every variable $x : A$ in Γ to a value $\Delta \vdash^{\mathsf{v}} V : A$.

Each type former, constructor, and substitution operation of CBPV has a semantic counterpart in the bimodel, which directly paraphrases the typing rules of Fig. 1 and the stack typing of Fig. 2. To obtain a simple encoding of variables in type theory we use de Bruijn indices [de Bruijn 1972],

with q denoting the topmost variable in the context, and p the substitution that drops it. Their interaction with substitutions is standard [Altenkirch and Kaposi 2016].

Machine states are pairs of computations and stacks. The transition relation is a family on two machine states with constructors witnessing this relation for every transition rule of the CK machine shown in Fig. 2.

The most unusual aspect of bimodels is the presence of *resolution functions*. These operations resolve any closed value at a sum or product value type into its constituent parts. For example,

$$\mathsf{resolve}_+(-) : \mathsf{Val}\ \emptyset\ (A_1 + A_2) \to \mathsf{Val}\ \emptyset\ A_1 + \mathsf{Val}\ \emptyset\ A_2$$

allows us to *always* resolve a closed value of type $A_1 + A_2$ to either a closed value of type A_1 or a closed value of type A_2. This abstracts the syntactic invariant that closed sum and product values expose an observable constructor. In a general bimodel the resolution functions are part of the structure: they provide exactly the observations of closed values needed to define the value relation.

We postulate some coherence equations specifying that the resolution functions act as expected on closed values, e.g. $\mathsf{resolve}_+(\mathsf{inl}(V)) = \mathsf{inj}_1(V)$. Defining the function $\mathsf{in} : \mathsf{Val}\ \emptyset\ A_1 + \mathsf{Val}\ \emptyset\ A_2 \to \mathsf{Val}\ \emptyset\ (A_1 + A_2)$ to map $\mathsf{inj}_1(V) \mapsto \mathsf{inl}(V)$ and $\mathsf{inj}_2(V) \mapsto \mathsf{inr}(V)$, these coherence equations are equivalent to $\mathsf{resolve}_+ \circ \mathsf{in} = \mathsf{id}$, exhibiting $\mathsf{Val}\ \emptyset\ A_1 + \mathsf{Val}\ \emptyset\ A_2$ as a retract of $\mathsf{Val}\ \emptyset\ (A_1 + A_2)$. We stop short of requiring closed values to have inductive structure, i.e. requiring an isomorphism $\mathsf{Val}\ \emptyset\ (A_1 + A_2) \cong \mathsf{Val}\ \emptyset\ A_1 + \mathsf{Val}\ \emptyset\ A_2$. However, such an isomorphism exists in both our examples.

Interpretation. Given a bimodel $\mathfrak{B}$, we define an *interpretation* of CBPV syntax into $\mathfrak{B}$, giving

$$\begin{array}{lll} [\![\Gamma]\!] : \mathsf{Ctx} & [\![A]\!] : \mathsf{VTy} & [\![\underline{C}]\!] : \mathsf{CTy} \\ [\![V]\!] : \mathsf{Val}\ [\![\Gamma]\!]\ [\![A]\!] & [\![M]\!] : \mathsf{Com}\ [\![\Gamma]\!]\ [\![\underline{C}]\!] & [\![K]\!] : \mathsf{Stk}\ [\![\Gamma]\!]\ [\![\underline{C}]\!]\ [\![\underline{D}]\!] \end{array}$$

for each context Γ, value type A, computation type $\underline{C}$, value $\Gamma \vdash^{\mathrm{v}} V : A$, computation $\Gamma \vdash^{\mathrm{c}} M : \underline{C}$, and stack $\Gamma \vdash K : \underline{C} \multimap \underline{D}$. These maps are defined by induction on the structure of typing derivations, using the operations of the bimodel to interpret each rule (`CK.Bimodel.Interpretation`).

We can prove that this interpretation is injective at first-order types.

Lemma 4.1 (Injectivity at σ, `[[]]ᵛ-fo-inj`). *If* $\vdash^{\mathrm{v}} V : \sigma$ *and* $\vdash^{\mathrm{v}} W : \sigma$ *and* $[\![V]\!] = [\![W]\!]$, *then* $V = W$.

The proof of this lemma relies crucially on the coherence equation for resolution functions.

4.1 The Syntactic Bimodel

The simplest example of a bimodel is the *syntactic bimodel* $\mathfrak{B}_{\mathrm{Syn}}$, which is given by the syntax of CBPV itself (`CK.Bimodel.SyntacticBimodel`). The sorts are the sets of contexts and types, $\mathsf{Val}\ \Gamma\ A$ is the set of value derivations $\Gamma \vdash^{\mathrm{v}} V : A$, and so on. All operations are the syntactic constructors, the transition relation is the CK machine transition, and the resolution functions are given by pattern-matching, as every closed sum and product value is headed by a constructor. Notice that we do not require a quotient for the syntax to form a bimodel, as the required substitution equalities are witnessed by the standard substitution lemmata.

It can be shown that the interpretation into the syntactic bimodel is the identity, so that $[\![A]\!] = A$, $[\![V]\!] = V$, $[\![M]\!] = M$, and so on. While in set theory this is simple, it is somewhat more involved in type theory and in the Agda formalisation, as it involves a series of coherence lemmata relating the inductive definition to the syntactic operations.

4.2 The Set-Theoretic Bimodel

Our second example is the *set-theoretic bimodel* $\mathfrak{B}_{\mathrm{Set}}$, which has a much more mathematical and denotational flavour (`CK.Bimodel.TypeBimodel`). We set Ctx = VTy = CTy = Set.[3] Values, computations, and substitutions are all defined as functions on the appropriate sets:

$$\mathsf{Val}\ \Gamma\ A \stackrel{\mathrm{def}}{=} \Gamma \to A \qquad \mathsf{Com}\ \Gamma\ C \stackrel{\mathrm{def}}{=} \Gamma \to C$$

$$\mathsf{Stk}\ \Gamma\ C\ D \stackrel{\mathrm{def}}{=} \Gamma \to C \to D \qquad \mathsf{Sub}\ \Delta\ \Gamma \stackrel{\mathrm{def}}{=} \Delta \to \Gamma$$

Similarly, types are interpreted by the corresponding set-theoretic construction:

$$\mathbf{1}^{\mathfrak{B}_{\mathrm{Set}}} \stackrel{\mathrm{def}}{=} \top \qquad A_1 +^{\mathfrak{B}_{\mathrm{Set}}} A_2 \stackrel{\mathrm{def}}{=} A_1 + A_2 \qquad A_1 \times^{\mathfrak{B}_{\mathrm{Set}}} A_2 \stackrel{\mathrm{def}}{=} A_1 \times A_2$$

$$F(A)^{\mathfrak{B}_{\mathrm{Set}}} \stackrel{\mathrm{def}}{=} A \qquad (A \to^{\mathfrak{B}_{\mathrm{Set}}} C) \stackrel{\mathrm{def}}{=} A \to C \qquad U(C)^{\mathfrak{B}_{\mathrm{Set}}} \stackrel{\mathrm{def}}{=} C$$

Observe that U and F are interpreted as the identity; this is the manifestation of the fact that thunking and returning are trivial operations in a pure language. The resolution functions are the usual case analysis and projection operations on disjoint unions and Cartesian products.

Contexts are interpreted as products, with $\varnothing$ the singleton set $\top$ and $\Gamma.A$ the product $\Gamma \times A$. Substitutions are functions $\Delta \to \Gamma$, acting on terms by composition. As the transition relation relates only *closed* states, a computation is essentially[4] an element $m \in C$ and a stack is a function $k : C \to D$; we set $(m, k) \rightsquigarrow (m', k')$ just if $k(m) = k'(m')$. Thus the transition relation is denotational equality between states, and the reduction rules follow immediately from the definitions.

4.3 Biorthogonal Logical Relations for Bimodels

We can now define a biorthogonal logical relation for any bimodel, and prove the fundamental lemma once for each arity. We treat the unary and binary cases; other finite arities are treated in a similar way. Both relations follow the same pattern:

(1) begin with a (unary or binary) pole $\perp\!\!\!\perp$ at each computation type;
(2) define a value relation by induction on value types, using the resolution functions;
(3) define a terminal computation relation;
(4) define a stack relation by orthogonality against the pole $\perp\!\!\!\perp$; and then
(5) define a computation relation by orthogonality against the pole $\perp\!\!\!\perp$.

Unary Logical Relations. Fix a bimodel $\mathfrak{B}$. As before, a *pole* on $\mathfrak{B}$ is a type-indexed predicate

$$\perp\!\!\!\perp : (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}}\ [\![\underline{C}]\!]_{\mathfrak{B}} \to \mathsf{Set}$$

We write $S \in \perp\!\!\!\perp_{\underline{C}}$ instead of $\perp\!\!\!\perp\ \underline{C}\ S$ (`CK.SemanticLR.Unary`). As before, we require that it be closed under inverse evaluation, viz. that $S' \in \perp\!\!\!\perp_{\underline{C}}$ and $S \rightsquigarrow S'$ imply $S \in \perp\!\!\!\perp_{\underline{C}}$.

Using the bimodel $\mathfrak{B}$ and the pole $\perp\!\!\!\perp$ we can now define predicates

$$\mathcal{R}^{v}_{A} : \mathsf{Val}\ \varnothing\ [\![A]\!]_{\mathfrak{B}} \to \mathsf{Set} \qquad \mathcal{R}^{t}_{\underline{C}} : \mathsf{Com}\ \varnothing\ [\![\underline{C}]\!]_{\mathfrak{B}} \to \mathsf{Set}$$

$$\mathcal{R}^{k}_{\underline{C} \multimap \underline{D}} : \mathsf{Stk}\ \varnothing\ [\![\underline{C}]\!]_{\mathfrak{B}}\ [\![\underline{D}]\!]_{\mathfrak{B}} \to \mathsf{Set} \qquad \mathcal{R}^{e}_{\underline{C}} : \mathsf{Com}\ \varnothing\ [\![\underline{C}]\!]_{\mathfrak{B}} \to \mathsf{Set}$$

on the semantic domains of closed values, 'terminals,' stacks, and computations, but indexed by the syntax of types [Plotkin 1973]. The defining clauses are given in Fig. 5. They are essentially the same as those of Fig. 3. Nevertheless, we are no longer able to induct on the syntax, which necessitates some circumlocutions.

[3] This is possible because contexts and types live in the larger universe Set_1.

[4] Modulo the empty context $\varnothing \stackrel{\mathrm{def}}{=} \top$.

$$\mathcal{R}^v_{\mathbf{1}}\ V \stackrel{\text{def}}{\equiv} \top$$

$$\mathcal{R}^v_{A_1+A_2}\ V \stackrel{\text{def}}{\equiv} (\exists V_1 : \mathsf{Val}\ \varnothing\ [\![A_1]\!].\ \mathsf{resolve}_+(V) = \mathsf{inj}_1(V_1)\ \wedge\ \mathcal{R}^v_{A_1}\ V_1) \vee (\exists V_2 : \mathsf{Val}\ \varnothing\ [\![A_2]\!].\ \mathsf{resolve}_+(V) = \mathsf{inj}_2(V_2)\ \wedge\ \mathcal{R}^v_{A_2}\ V_2)$$

$$\mathcal{R}^v_{A_1\times A_2}\ V \stackrel{\text{def}}{\equiv} \exists V_1 : \mathsf{Val}\ \varnothing\ [\![A_1]\!],\ V_2 : \mathsf{Val}\ \varnothing\ [\![A_2]\!].\ \mathsf{resolve}_\times(V) = (V_1, V_2)\ \wedge\ \mathcal{R}^v_{A_1}\ V_1\ \wedge\ \mathcal{R}^v_{A_2}\ V_2$$

$$\mathcal{R}^v_{U\underline{C}}\ V \stackrel{\text{def}}{\equiv} \mathcal{R}^c_{\underline{C}}\ \mathsf{force}^{\mathfrak{B}}(V)$$

$$\mathcal{R}^t_{FA}\ M \stackrel{\text{def}}{\equiv} \exists V : \mathsf{Val}\ \varnothing\ [\![A]\!].\ M = \mathsf{return}^{\mathfrak{B}}(V)\ \wedge\ \mathcal{R}^v_A\ V$$

$$\mathcal{R}^t_{A\to\underline{C}}\ M \stackrel{\text{def}}{\equiv} \exists N : \mathsf{Com}\ (\varnothing\centerdot[\![A]\!])\ [\![\underline{C}]\!].\ M = \lambda^{\mathfrak{B}}(N)\ \wedge\ \forall V.\ \mathcal{R}^v_A\ V \Rightarrow \mathcal{R}^c_{\underline{C}}\ N[\epsilon\cdot V]^{\mathsf{c}}$$

$$\mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K \stackrel{\text{def}}{\equiv} \forall M : \mathsf{Com}\ \varnothing\ [\![\underline{C}]\!].\mathcal{R}^t_{\underline{C}}\ M\ \Rightarrow\ (M,K) \in \perp\!\!\!\perp_{\underline{D}}$$

$$\mathcal{R}^c_{\underline{C}}\ M \stackrel{\text{def}}{\equiv} \forall \underline{D},\ K : \mathsf{Stk}\ \varnothing\ [\![\underline{C}]\!]\ [\![\underline{D}]\!].\ \mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K \Rightarrow (M,K) \in \perp\!\!\!\perp_{\underline{D}}$$

Fig. 5. Unary logical relation for bimodels (R^{v}, R^{t}, R^{k}, R^{c}).

To define the predicate on values we use the resolution functions that come with the bimodel to resolve every value at sum and product types, and then apply the logical relation as before. For sums, the branch selected by $\mathsf{resolve}_+(-)$ determines which component relation must hold; for products, $\mathsf{resolve}_\times(-)$ returns both components. For thunks we have no choice but to apply force to the value to obtain a computation, and ask that the logical relation be satisfied by this computation.

Similarly, 'terminal' computations do not exist in a bimodel as a semantic entity. Thus, to define a predicate we once more use an existential quantifier to carve out those computations that are in the image of either return or λ. The rest of the definition is as before, but phrased in terms of the bimodel operations rather than the syntax of CBPV.

For stacks and computations, we use the same orthogonality clauses as before, but with the bimodel operations instead of the syntax again.

We inductively define a predicate $\mathcal{R}^\Gamma : \mathsf{Sub}\ \varnothing\ [\![\Gamma]\!] \to \mathsf{Set}$ on closed substitutions (R^{e}), carving out those that are constructed using only values in the value relation. It is given by the rules

$$\frac{}{\mathcal{R}^{\emptyset}\ \epsilon} \qquad\qquad \frac{\mathcal{R}^\Gamma\ \sigma \qquad \mathcal{R}^v_A\ V}{\mathcal{R}^{\Gamma\centerdot A}\ (\sigma\cdot V)}$$

A simple induction on typing can then show that

Lemma 4.2 (Fundamental lemma, unary, `rel-val`, `rel-com`). *Let $\mathfrak{B}$ be a bimodel and $\perp\!\!\!\perp$ a pole.*
(1) *If $\Gamma \vdash^{\mathsf{v}} V : A$ and $\mathcal{R}^\Gamma\ \gamma$ then $\mathcal{R}^v_A\ [\![V]\!]\,[\gamma]^{\mathsf{v}}$.*
(2) *If $\Gamma \vdash^{\mathsf{c}} M : \underline{C}$ and $\mathcal{R}^\Gamma\ \gamma$ then $\mathcal{R}^c_{\underline{C}}\ [\![M]\!]\,[\gamma]^{\mathsf{c}}$.*

The proof is much the same as that of Lemma 3.2, but phrased in terms of bimodel operations.

Binary Logical Relations. The same approach can be immediately adapted to binary logical relations (`CK.SemanticLR.Binary`). Let $\mathfrak{B}_1$ and $\mathfrak{B}_2$ be bimodels with interpretations $[\![-]\!]_{\mathfrak{B}_1}$ and $[\![-]\!]_{\mathfrak{B}_2}$. We use the same notation for each bimodel, but subscript it to denote which one it comes from, e.g. ϵ_1 and ϵ_2 for the empty substitutions, $\mathsf{inl}^{\mathfrak{B}_1}$ and $\mathsf{inl}^{\mathfrak{B}_2}$ for the injections, and so on.

$$V_1 \mathrel{\mathcal{R}^v_{\mathbf{1}}} V_2 \stackrel{\text{def}}{\equiv} \top$$

$$\begin{aligned} V_1 \mathrel{\mathcal{R}^v_{A_1+A_2}} V_2 \stackrel{\text{def}}{\equiv}\ & (\exists V_1' : \mathsf{Val}\ \varnothing_1\ [\![A_1]\!].\ \exists V_2' : \mathsf{Val}\ \varnothing_2\ [\![A_1]\!]. \\ & \quad \mathsf{resolve}_{+,1}(V_1) = \mathsf{inj}_1(V_1') \wedge \mathsf{resolve}_{+,2}(V_2) = \mathsf{inj}_1(V_2') \ \wedge\ V_1' \mathrel{\mathcal{R}^v_{A_1}} V_2') \vee \\ & (\exists V_1' : \mathsf{Val}\ \varnothing_1\ [\![A_2]\!].\ \exists V_2' : \mathsf{Val}\ \varnothing_2\ [\![A_2]\!]. \\ & \quad \mathsf{resolve}_{+,1}(V_1) = \mathsf{inj}_2(V_1') \wedge \mathsf{resolve}_{+,2}(V_2) = \mathsf{inj}_2(V_2') \ \wedge\ V_1' \mathrel{\mathcal{R}^v_{A_2}} V_2') \end{aligned}$$

$$\begin{aligned} V_1 \mathrel{\mathcal{R}^v_{A_1\times A_2}} V_2 \stackrel{\text{def}}{\equiv}\ & \exists W_1 : \mathsf{Val}\ \varnothing_1\ [\![A_1]\!],\ W_1' : \mathsf{Val}\ \varnothing_1\ [\![A_2]\!],\ W_2 : \mathsf{Val}\ \varnothing_2\ [\![A_1]\!],\ W_2' : \mathsf{Val}\ \varnothing_2\ [\![A_2]\!]. \\ & \quad \mathsf{resolve}_{\times,1}(V_1) = (W_1, W_1') \wedge \mathsf{resolve}_{\times,2}(V_2) = (W_2, W_2') \ \wedge\ W_1 \mathrel{\mathcal{R}^v_{A_1}} W_2 \ \wedge\ W_1' \mathrel{\mathcal{R}^v_{A_2}} W_2' \end{aligned}$$

$$V_1 \mathrel{\mathcal{R}^v_{U\underline{C}}} V_2 \stackrel{\text{def}}{\equiv} \mathsf{force}^{\mathfrak{B}_1}(V_1) \mathrel{\mathcal{R}^c_{\underline{C}}} \mathsf{force}^{\mathfrak{B}_2}(V_2)$$

$$\begin{aligned} T_1 \mathrel{\mathcal{R}^t_{FA}} T_2 \stackrel{\text{def}}{\equiv}\ & \exists V_1 : \mathsf{Val}\ \varnothing_1\ [\![A]\!].\ \exists V_2 : \mathsf{Val}\ \varnothing_2\ [\![A]\!]. \\ & \quad T_1 = \mathsf{return}^{\mathfrak{B}_1}(V_1) \ \wedge\ T_2 = \mathsf{return}^{\mathfrak{B}_2}(V_2) \ \wedge\ V_1 \mathrel{\mathcal{R}^v_A} V_2 \end{aligned}$$

$$\begin{aligned} T_1 \mathrel{\mathcal{R}^t_{A\to\underline{C}}} T_2 \stackrel{\text{def}}{\equiv}\ & \exists N_1 : \mathsf{Com}\ (\varnothing_1 \centerdot [\![A]\!])\ [\![\underline{C}]\!].\ \exists N_2 : \mathsf{Com}\ (\varnothing_2 \centerdot [\![A]\!])\ [\![\underline{C}]\!]. \\ & \quad T_1 = \lambda^{\mathfrak{B}_1}(N_1) \ \wedge\ T_2 = \lambda^{\mathfrak{B}_2}(N_2) \ \wedge\ \forall V_1, V_2.\ V_1 \mathrel{\mathcal{R}^v_A} V_2 \Rightarrow N_1[\epsilon_1 \cdot V_1]^{\mathsf{c}} \mathrel{\mathcal{R}^c_{\underline{C}}} N_2[\epsilon_2 \cdot V_2]^{\mathsf{c}} \end{aligned}$$

$$K_1 \mathrel{\mathcal{R}^k_{\underline{C}\multimap\underline{D}}} K_2 \stackrel{\text{def}}{\equiv} \forall M_1 : \mathsf{Com}\ \varnothing_1\ [\![\underline{C}]\!].\ \forall M_2 : \mathsf{Com}\ \varnothing_2\ [\![\underline{C}]\!].\ M_1 \mathrel{\mathcal{R}^t_{\underline{C}}} M_2 \ \Rightarrow\ ((M_1, K_1), (M_2, K_2)) \in \perp\!\!\!\perp_{\underline{D}}$$

$$\begin{aligned} M_1 \mathrel{\mathcal{R}^c_{\underline{C}}} M_2 \stackrel{\text{def}}{\equiv}\ & \forall \underline{D}.\ \forall K_1 : \mathsf{Stk}\ \varnothing_1\ [\![\underline{C}]\!]\ [\![\underline{D}]\!].\ \forall K_2 : \mathsf{Stk}\ \varnothing_2\ [\![\underline{C}]\!]\ [\![\underline{D}]\!]. \\ & \quad K_1 \mathrel{\mathcal{R}^k_{\underline{C}\multimap\underline{D}}} K_2 \Rightarrow ((M_1, K_1), (M_2, K_2)) \in \perp\!\!\!\perp_{\underline{D}} \end{aligned}$$

Fig. 6. Binary logical relation for bimodels (R^{v}, R^{t}, R^{k}, R^{c}).

A *binary pole* on $\mathfrak{B}_1$ and $\mathfrak{B}_2$ is a type-indexed relation

$$\perp\!\!\!\perp : (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_1}\ [\![\underline{C}]\!]_{\mathfrak{B}_1} \to \mathsf{State}^{\mathfrak{B}_2}\ [\![\underline{C}]\!]_{\mathfrak{B}_2} \to \mathsf{Set}$$

We normally write $(S_1, S_2) \in \perp\!\!\!\perp_{\underline{C}}$ instead of $\perp\!\!\!\perp\ \underline{C}\ S_1\ S_2$. We require that the pole be closed under inverse evaluation on both sides, viz. that $(S_1', S_2) \in \perp\!\!\!\perp_{\underline{C}}$ and $S_1 \rightsquigarrow S_1'$ imply $(S_1, S_2) \in \perp\!\!\!\perp_{\underline{C}}$, and likewise for the second component.

Using the bimodels $\mathfrak{B}_1$, $\mathfrak{B}_2$ and the pole $\perp\!\!\!\perp$ we can now define binary relations

$$\begin{aligned} \mathcal{R}^v_A &: \mathsf{Val}\ \varnothing_1\ [\![A]\!]_{\mathfrak{B}_1} \to \mathsf{Val}\ \varnothing_2\ [\![A]\!]_{\mathfrak{B}_2} \to \mathsf{Set} \\ \mathcal{R}^t_{\underline{C}} &: \mathsf{Com}\ \varnothing_1\ [\![\underline{C}]\!]_{\mathfrak{B}_1} \to \mathsf{Com}\ \varnothing_2\ [\![\underline{C}]\!]_{\mathfrak{B}_2} \to \mathsf{Set} \\ \mathcal{R}^k_{\underline{C}\multimap\underline{D}} &: \mathsf{Stk}\ \varnothing_1\ [\![\underline{C}]\!]_{\mathfrak{B}_1}\ [\![\underline{D}]\!]_{\mathfrak{B}_1} \to \mathsf{Stk}\ \varnothing_2\ [\![\underline{C}]\!]_{\mathfrak{B}_2}\ [\![\underline{D}]\!]_{\mathfrak{B}_2} \to \mathsf{Set} \\ \mathcal{R}^c_{\underline{C}} &: \mathsf{Com}\ \varnothing_1\ [\![\underline{C}]\!]_{\mathfrak{B}_1} \to \mathsf{Com}\ \varnothing_2\ [\![\underline{C}]\!]_{\mathfrak{B}_2} \to \mathsf{Set} \end{aligned}$$

between pairs of interpretations of closed values, terminals, stacks, and computations, again indexed by the syntax of types. The definitions are given in Fig. 6; they are analogous to the unary case.

We inductively define a binary relation $\mathcal{R}^\Gamma : \mathsf{Sub}\ \varnothing_1\ [\![\Gamma]\!]_{\mathfrak{B}_1} \to \mathsf{Sub}\ \varnothing_2\ [\![\Gamma]\!]_{\mathfrak{B}_2} \to \mathsf{Set}$ on pairs of closed substitutions, carving out those that are constructed using only pairs in the value relation:

$$\frac{}{\epsilon_1 \mathrel{\mathcal{R}^\emptyset} \epsilon_2} \qquad\qquad \frac{\sigma_1 \mathrel{\mathcal{R}^\Gamma} \sigma_2 \qquad V_1 \mathrel{\mathcal{R}^v_A} V_2}{(\sigma_1 \cdot V_1) \mathrel{\mathcal{R}^{\Gamma\centerdot A}} (\sigma_2 \cdot V_2)}$$

Again, by a simple induction on typing derivations,

Lemma 4.3 (Fundamental lemma, binary, `rel-val`, `rel-com`).
Let $\mathfrak{B}_1$ and $\mathfrak{B}_2$ be bimodels and $\perp\!\!\!\perp$ a binary pole.
(1) *If* $\Gamma \vdash^{\mathrm{v}} V : A$ *and* $\gamma_1 \; \mathcal{R}^{\Gamma} \; \gamma_2$ *then* $[\![V]\!]_{\mathfrak{B}_1}[\gamma_1]^{\mathrm{v}} \; \mathcal{R}^{v}_{A} \; [\![V]\!]_{\mathfrak{B}_2}[\gamma_2]^{\mathrm{v}}$.
(2) *If* $\Gamma \vdash^{\mathrm{c}} M : \underline{C}$ *and* $\gamma_1 \; \mathcal{R}^{\Gamma} \; \gamma_2$ *then* $[\![M]\!]_{\mathfrak{B}_1}[\gamma_1]^{\mathrm{c}} \; \mathcal{R}^{c}_{\underline{C}} \; [\![M]\!]_{\mathfrak{B}_2}[\gamma_2]^{\mathrm{c}}$.

The proof is essentially the same as that of Lemma 4.2, but quantifying over pairs of related substitutions and using backward closure of the pole on both sides.

5 Applications of Bimodels

The logical relations of Section 4 are generic in the choice of bimodel and pole. Instantiating them with different choices yields a range of consequences from the same fundamental lemmata.

These are not separate proofs by logical relations, but rather applications of the unary and binary fundamental lemmata (Lemmas 4.2 and 4.3). Each application is obtained by picking an instantiation of the bimodel(s) and a pole: the unary logical relation on the syntactic bimodel yields a proof of **canonicity** for the CK machine (Section 5.1); the binary logical relation between the syntactic and set-theoretic bimodels yields **adequacy** of denotational semantics (Section 5.2); two copies of the syntactic bimodel yield **operational extensionality**, which we characterize as a CIU approximation (Section 5.3); and two copies of the set-theoretic bimodel yield **internal full abstraction**, where the pole-induced equivalence coincides with mathematical equality (Section 5.4).

5.1 Canonicity

Consider the following unary pole on the syntactic bimodel (`CK.SemanticLR.Canonicity`):

$$\begin{array}{lcl} \perp\!\!\!\perp_{\mathsf{Can}} & : & (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Syn}}}\, [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Syn}}} \to \mathsf{Set} \\ \perp\!\!\!\perp_{\mathsf{Can}}\ (FA)\ S & \stackrel{\mathrm{def}}{=} & \exists V : A.\ S \rightsquigarrow^{*} (\mathsf{return}\ V, \odot) \\ \perp\!\!\!\perp_{\mathsf{Can}}\ (A \to \underline{C})\ _ & \stackrel{\mathrm{def}}{=} & \bot \end{array}$$

This pole is defined by induction on types. A CK machine $\vdash \langle M \mid K \rangle : FA$ is in this pole (as (M, K)) if it returns a value of the appropriate type. Machines of type $A \to \underline{C}$ are *never* in this pole. Thus, the pole only contains machines which eventually return a value. This is an arbitrary choice: we could equally have included some machines of function type, e.g. those halting with a λ-abstraction. However, this would have led to a canonicity theorem like that of Section 3, so we pick a different, weaker pole to show the flexibility of this approach.

$\perp\!\!\!\perp_{\mathsf{Can}}$ is evidently closed under inverse evaluation, by a simple induction on the definition of $\rightsquigarrow^{*}$. We therefore obtain unary logical relations $\mathcal{R}^{v}_{A}$ and $\mathcal{R}^{c}_{\underline{C}}$ at each value type A and computation type $\underline{C}$. By Lemma 4.2 we know that $\mathcal{R}^{v}_{A}\ [\![V]\!]$ and $\mathcal{R}^{c}_{\underline{C}}\ [\![M]\!]$ for all closed $\vdash^{\mathrm{v}} V : A$ and $\vdash^{\mathrm{c}} M : \underline{C}$.

Before obtaining our result we also have to show that $\mathcal{R}^{k}_{FA \multimap FA}\ \odot$ for all value types A. That requires showing that for every $\vdash^{\mathrm{c}} M : FA$ for which $\mathcal{R}^{t}_{FA}\ M$ we have $(M, \odot) \in \perp\!\!\!\perp_{\mathsf{Can}}$. But $\mathcal{R}^{t}_{FA}\ M$ in the syntactic model implies that $M = \mathsf{return}\ V$ for some value V of type A, so $(M, \odot)$ is in the pole. Note that it is *never* true that $\mathcal{R}^{k}_{(A \to \underline{C}) \multimap (A \to \underline{C})}\ \odot$, because the pole contains no machines of function type. Thus it is *not* generally true that the fundamental lemma holds of stacks. Putting $\mathcal{R}^{c}_{FA}\ [\![M]\!]$ together with $\mathcal{R}^{k}_{FA \multimap FA}\ \odot$, we have that for any $\vdash^{\mathrm{c}} M : FA$, $(M, \odot) \in \perp\!\!\!\perp_{\mathsf{Can}}$, i.e.

Theorem 5.1 (Canonicity for CK, `canonicity`). *If* $\vdash^{\mathrm{c}} M : FA$, *then* $\langle M \mid \odot \rangle \rightsquigarrow^{*} \langle \mathsf{return}\ V \mid \odot \rangle$ *for some closed value* $V : A$.

5.2 Adequacy for CK

In this section we prove that the set-theoretic bimodel is *adequate* for the CK machine [Milner 1975]: if the set-theoretic bimodel predicts that a term $M : FA$ returns a value $V : A$, then the

CK machine loaded with M and the empty stack $\odot$ must in fact compute this value. We do so by instantiating the binary logical relation on the syntactic bimodel and the set-theoretic bimodel with a binary pole (`CK.SemanticLR.Adequacy`).

The formal statement of adequacy depends critically on what is considered *observable*. It cannot hold at all value types, because they contain thunks: even if $[\![M]\!] = [\![M']\!] \in [\![FU(A \to \underline{C})]\!]$ are equal computations returning thunks of functions, the CK machine may still produce syntactically different values from M and M'. Thus the CK machine cannot be adequate at thunks. Adequacy can only hold where distinct syntactic values have distinct denotations. The largest such class in the set-theoretic model is the *first-order types* of Fig. 1; indeed, Plotkin's original adequacy result for PCF [1977] concerns the natural numbers, a first-order type.

With that in mind, we instantiate the binary logical relation between the syntactic bimodel and the set-theoretic bimodel, and define the following pole

$$
\begin{array}{lcl}
\perp\!\!\!\perp_{\mathsf{Adeq}} & : & (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Set}}}\, [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Set}}} \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Syn}}}\, [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Syn}}} \to \mathsf{Set} \\
\perp\!\!\!\perp_{\mathsf{Adeq}}\ (FA)\ S_1\ S_2 & \stackrel{\text{def}}{=} & \forall V : A.\ S_1 \rightsquigarrow^* ([\![\mathsf{return}\ V]\!], \odot) \to S_2 \rightsquigarrow^* (\mathsf{return}\ V, \odot) \\
\perp\!\!\!\perp_{\mathsf{Adeq}}\ _\ _\ _ & \stackrel{\text{def}}{=} & \bot
\end{array}
$$

This pole says that if the 'semantic machine' S_1 returns $[\![\mathsf{return}\ V]\!]$, then the 'syntactic machine' S_2 must return $\mathsf{return}\ V$. This need not hold when V contains a thunk of function type, so the theorem below restricts observability to first-order values. $\perp\!\!\!\perp_{\mathsf{Adeq}}$ is closed under inverse evaluation, so by Lemma 4.3 we have $[\![V]\!]_{\mathfrak{B}_{\mathsf{Set}}}\ \mathcal{R}^v_A\ [\![V]\!]_{\mathfrak{B}_{\mathsf{Syn}}}$ and $[\![M]\!]_{\mathfrak{B}_{\mathsf{Set}}}\ \mathcal{R}^c_{\underline{C}}\ [\![M]\!]_{\mathfrak{B}_{\mathsf{Syn}}}$ for all closed $\vdash^{\mathsf{v}} V : A$ and $\vdash^{\mathsf{c}} M : \underline{C}$.

The next step is to show that $\odot\ \mathcal{R}^k_{F\sigma \multimap F\sigma}\ \odot$ for all first-order value types σ, but we immediately encounter issues. We have to show that whenever $m\ \mathcal{R}^t_{F\sigma}\ M$, the pair $((m, \odot), (M, \odot))$ lies in $\perp\!\!\!\perp_{\mathsf{Adeq}}$. Unfolding $m\ \mathcal{R}^t_{F\sigma}\ M$ gives $m = \mathsf{return}^{\mathfrak{B}_{\mathsf{Set}}}(w)$ for some $w : [\![\sigma]\!]$ and $M = \mathsf{return}\ W$ for some $W : \sigma$ with $w\ \mathcal{R}^v_\sigma\ W$. It transpires that this $\mathcal{R}^v_\sigma$ has a special property at first-order types [Kavvos 2025].

Lemma 5.2 (Graph Lemma for CK, `R^v-sound`). *If $v\ \mathcal{R}^v_\sigma\ V$ then $v = [\![V]\!]$.*

We can therefore immediately deduce that $w = [\![W]\!]$. Unfolding the definitions, it remains to prove that, for any $V : \sigma$, if $w = [\![V]\!]$ then $\langle \mathsf{return}\ W \mid \odot \rangle \rightsquigarrow^* \langle \mathsf{return}\ V \mid \odot \rangle$, i.e. $W = V$ syntactically. But then $[\![W]\!] = w = [\![V]\!]$, and by Lemma 4.1 we have $W = V$ syntactically. Hence:

Theorem 5.3 (Adequacy for CK, `adequacy`). *Let σ be a first-order value type, $\vdash^{\mathsf{c}} M : F\sigma$, and $\vdash^{\mathsf{v}} V : \sigma$. If $[\![M]\!]_{\mathfrak{B}_{\mathsf{Set}}} = [\![\mathsf{return}\ V]\!]_{\mathfrak{B}_{\mathsf{Set}}}$ in the set-theoretic bimodel, then $\langle M \mid \odot \rangle \rightsquigarrow^* \langle \mathsf{return}\ V \mid \odot \rangle$.*

Proof. By Lemma 4.3 we have that $[\![M]\!]_{\mathfrak{B}_{\mathsf{Set}}}\ \mathcal{R}^c_{F\sigma}\ [\![M]\!]_{\mathfrak{B}_{\mathsf{Syn}}}$. As $\odot\ \mathcal{R}^k_{F\sigma \multimap F\sigma}\ \odot$, we obtain that $((([\![M]\!]_{\mathfrak{B}_{\mathsf{Set}}}, \odot), ([\![M]\!]_{\mathfrak{B}_{\mathsf{Syn}}}, \odot)) \in \perp\!\!\!\perp_{\mathsf{Adeq}}$. But as $[\![M]\!]_{\mathfrak{B}_{\mathsf{Syn}}} = M$, this implies the desired result. □

5.3 Operational Extensionality and Closed Instantiations of Use (CIU)

Constructing a logical relation between two copies of the same bimodel often has the flavour of an *approximation* between programs. We close this section on bimodels by showing that the relation obtained between two copies of the *syntactic* bimodel (and an appropriate pole) coincides with *closed-instantiations-of-use (CIU) approximation*, a standard notion of approximation in operational semantics, due to Mason and Talcott [1991] (`CK.SemanticLR.CIUEquiv`). Showing that a logical relation is exactly CIU is the usual stepping stone towards characterizing contextual equivalence; here it arises as an instance of our generic logical relation.

First, define the binary pole

$$\begin{array}{lcl} \perp\!\!\!\perp_{\mathsf{CIU}} & : & (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Syn}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Syn}}} \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Syn}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Syn}}} \to \mathsf{Set} \\ \perp\!\!\!\perp_{\mathsf{CIU}}\ (FA)\ S_1\ S_2 & \stackrel{\text{def}}{=} & \mathsf{FO}(A) \to \forall V : A.\ S_1 \rightsquigarrow^* (\mathsf{return}\ V, \odot) \to S_2 \rightsquigarrow^* (\mathsf{return}\ V, \odot) \\ \perp\!\!\!\perp_{\mathsf{CIU}}\ _\ _\ _ & \stackrel{\text{def}}{=} & \bot \end{array}$$

Notice that this predicate guards the observation by $\mathsf{FO}(A)$. The reason is the same as before: we wish to define observable behaviour as that observable through first-order values, as we cannot reliably observe thunks. It is an implication, not a conjunction, so the clause holds vacuously at non-first-order types. Proving that this is a pole relies on determinism of the CK machine, used to transfer the return observation across inverse steps. We obtain a logical relation $\mathcal{R}^c_{\underline{C}}$, which we can think of as an approximation relation. We extend it to open terms through substitution: for $\Gamma \vdash^{\mathsf{c}} M, N : \underline{C}$ we let $M\ \mathcal{R}^c_{\underline{C}}\ N$ if $M[\gamma]\ \mathcal{R}^c_{\underline{C}}\ N[\gamma]$ for all closing substitutions $\gamma : \Gamma$.

The *closed-instantiations-of-use (CIU)* approximation relation is defined as it says on the tin: a computation approximates another if, for whichever closing substitution ('closed') we apply and whichever stack ('use') we couple it with, the first computation's returning a first-order value implies the second does too. In symbols, for any two computations $\Gamma \vdash^{\mathsf{c}} M, N : \underline{C}$ we let

$$\begin{array}{rcl} M \precsim^{\mathsf{ciu}}_{\underline{C}} N & \stackrel{\text{def}}{\equiv} & \forall A.\ \mathsf{FO}(A) \to \forall \gamma : \Gamma,\ \vdash K : \underline{C} \multimap FA,\ V : A. \\ & & \langle M[\gamma] \mid K \rangle \rightsquigarrow^* \langle \mathsf{return}\ V \mid \odot \rangle \to \langle N[\gamma] \mid K \rangle \rightsquigarrow^* \langle \mathsf{return}\ V \mid \odot \rangle \end{array}$$

To prove that this coincides with the logical relation we first prove a series of lemmata. First, we show that any two first-order values related by the logical relation must be syntactically identical.

Lemma 5.4 (R^{v}-fo-≡). *For any first-order type σ, if $V_1\ \mathcal{R}^v_\sigma\ V_2$ then $V_1 = V_2$.*

This is shown by a simple induction on first-order types. It can be used to prove that $\odot\ \mathcal{R}^k_{FA \multimap FA}\ \odot$ for any value type A. Following that, we use the proof strategy of Pitts [2010]. First, we show that the logical relation is a 'right CIU module', in the sense that

Lemma 5.5 ($\lesssim^{\mathsf{LR}}$-o-$\precsim^{\mathsf{CIU}}$). *If $M\ \mathcal{R}^c_{\underline{C}}\ N$ and $N \precsim^{\mathsf{ciu}} N'$ then $M\ \mathcal{R}^c_{\underline{C}}\ N'$.*

The fundamental lemma then implies $M\ \mathcal{R}^c_{\underline{C}}\ M$ for all M. By this property we have that $M \precsim^{\mathsf{ciu}} N$ implies $M\ \mathcal{R}^c_{\underline{C}}\ N$, i.e. $\precsim^{\mathsf{ciu}}$ is contained in $\mathcal{R}^c_{\underline{C}}$.

It is easy to bootstrap the fact that $\odot\ \mathcal{R}^k_{FA \multimap FA}\ \odot$ into an inductive proof that $K\ \mathcal{R}^k_{\underline{C} \multimap FA}\ K$ for any stack K. Given two computations $\Gamma \vdash^{\mathsf{c}} M, N : \underline{C}$, assume that $M\ \mathcal{R}^c_{\underline{C}}\ N$, and suppose we are given a $\gamma : \Gamma$ and a stack $\vdash K : \underline{C} \multimap F\sigma$. By definition this means that $M[\gamma]\ \mathcal{R}^c_{\underline{C}}\ N[\gamma]$, and hence that $((M[\gamma], K), (N[\gamma], K)) \in \perp\!\!\!\perp_{\mathsf{CIU}}$. Therefore $M \precsim^{\mathsf{ciu}}_{\underline{C}} N$. This concludes the proof that

Theorem 5.6 (Logical approximation equals CIU, $\lesssim^{\mathsf{LR}} \Leftrightarrow \precsim^{\mathsf{CIU}}$). *$\mathcal{R}^c_{\underline{C}}$ and $\precsim^{\mathsf{ciu}}_{\underline{C}}$ coincide.*

5.4 Internal Full Abstraction

The previous section read a pole as a notion of program equivalence; the same reading may be applied to any bimodel. A binary pole $\perp\!\!\!\perp$ supplies the observational equality that bimodels otherwise lack: two closed computations are equivalent when no related pair of stacks tells them apart.

In the terminology of Stark [1996], a bimodel is *internally fully abstract* when the induced equivalence is already (metatheoretic) equality (`CK.SemanticLR.IFA`).

Definition 5.7 (Internal full abstraction). Let $\mathfrak{B}$ be a bimodel, $\perp\!\!\!\perp$ a binary pole on two copies of $\mathfrak{B}$. $(\mathfrak{B}, \perp\!\!\!\perp)$ is *internally fully abstract* when the logical relations induced by $\perp\!\!\!\perp$ coincide with equality:

$$v_1\ \mathcal{R}^v_A\ v_2 \iff v_1 = v_2, \qquad f_1\ \mathcal{R}^c_{\underline{C}}\ f_2 \iff f_1 = f_2, \qquad k_1\ \mathcal{R}^k_{\underline{C} \multimap \underline{D}}\ k_2 \iff k_1 = k_2.$$

The definition asks for an equivalence in both directions, but only one of them carries weight. That every semantic object is related to itself is mere reflexivity. The content of internal full abstraction is the converse, viz. that related objects are equal ($R \subseteq =$). This says that the observations encoded in the pole suffice to tell any two distinct semantic objects apart. Together, the two directions say that the pole's observations *determine* the model's equality: two objects are equal exactly when no observation distinguishes them.

We can show that the set-theoretic bimodel is internally fully abstract when equipped with the *equality pole* (`CK.SemanticLR.TypeBimodelIFA`).

$$\begin{array}{lcl} \perp\!\!\!\perp_{\mathsf{Eq}} & : & (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Set}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Set}}} \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Set}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Set}}} \to \mathsf{Set} \\ \perp\!\!\!\perp_{\mathsf{Eq}}\ \underline{C}\ (f_1, k_1)\ (f_2, k_2) & \stackrel{\text{def}}{=} & k_1(f_1) = k_2(f_2) \end{array}$$

Since the empty context is the singleton, a closed computation f_i is an element of $[\![\underline{D}]\!]$ for some intermediate computation type $\underline{D}$, and a closed stack k_i is a function $[\![\underline{D}]\!] \to [\![\underline{C}]\!]$; their application $k_i(f_i) \in [\![\underline{C}]\!]$ denotes the state, and as reduction is interpreted as equality, closure under inverse evaluation is immediate.

Theorem 5.8 (`R^V-implies-≡`, `R^C-implies-≡`, `R^K-to-≡`). *$(\mathfrak{B}_{Set}, \perp\!\!\!\perp_{Eq})$ is internally fully abstract.*

This is established by a straightforward induction on types, using function extensionality. Internal full abstraction is akin to 'classical' full abstraction [Plotkin 1977], not its opposite: both ask that denotational equality coincide with an observational equivalence. They differ only in the observations—a fixed pole $\perp\!\!\!\perp$ here, quantification over all program contexts there—and agree precisely when the pole captures what a context can observe.

6 The CEK Machine

So far we have focused on the CK machine, which uses substitution to implement functions. This is an attractive and conceptually clean presentation but it is very far from how real implementations work. In this section we introduce the **CEK machine**, which uses environments and closures to implement substitution [Curien 1991; Felleisen and Friedman 1987].

Our CEK machine for CBPV is given in Fig. 7; we now explain its moving parts.

Value Closures, Computation Closures, and Environments. In CEK machines, variables are indirections that need not be eliminated by substitution, but only unfolded when necessary. This is achieved by temporarily storing their values in an *environment*, i.e. a map that gives meaning to free variables. Terms are then represented as *closures*, i.e. tuples of a term and the environment in which it is meant to be evaluated. In practice, closures and environments are represented in real machines by pointers, which is much more efficient than substitution.

In CBPV we have two kinds of closures: *value closures* and *computation closures*. As variables stand for values, the environment binds only values.

A *value closure* $v = (V_c, E)$ pairs a *canonical* value V_c, viz. one of $\langle\rangle$, a pair, an injection, or a thunk, along with an environment E. The grammar forbids storing a variable in a closure: *the value in every value closure must be headed by a constructor*, not by a variable. This invariant is maintained at every machine step, as explained below, and has two motivations. The first is efficiency: looking up a value in the environment immediately yields information, without requiring further indirections and dereferencing. The second is that it allows for a parsimonious presentation of an *environment bimodel* with only a few coherence equations, as described in Section 7.

Environments are inductively built from the empty environment $\varnothing$ by extension $E[x \mapsto v]$, which binds x to the value closure v. Finally, a *computation closure* $c = \langle M, E\rangle$ pairs a computation M with its environment E.

$\boxed{V_c}$ **Canonical values**

$$V_c \quad ::= \quad \langle\rangle \mid (V_1, V_2) \mid \mathsf{inl}(V) \mid \mathsf{inr}(V) \mid \mathsf{thunk}\ M$$

$\boxed{v, w}$ **Value closures**

$$v, w \quad ::= \quad (V_c,\ E)$$

$\boxed{c}$ **Computation closures**

$$c \quad ::= \quad \langle M,\ E\rangle$$

$\boxed{c_T}$ **Terminal closures**

$$c_T \quad ::= \quad \langle T,\ E\rangle$$

$\boxed{E}$ **Environments**

$$E \quad ::= \quad \varnothing \mid E[x \mapsto v]$$

$\boxed{K}$ **Stacks**

$$K \quad ::= \quad \odot \mid v :: K \mid \mathsf{to}(x.\, N,\ E) :: K$$

$\boxed{S}$ **Machines**

$$S \quad ::= \quad \langle c \mid K\rangle$$

$\boxed{S_T}$ **Terminal machines**

$$S_T \quad ::= \quad \langle c_T \mid \odot\rangle$$

$\boxed{S \rightsquigarrow S'}$ **Transition rules**

$$\langle\langle M(V),\ E\rangle \mid K\rangle \rightsquigarrow \langle\langle M,\ E\rangle \mid (V,\ E) :: K\rangle$$
$$\langle\langle \lambda x.\, M,\ E\rangle \mid v :: K\rangle \rightsquigarrow \langle\langle M,\ E[x \mapsto v]\rangle \mid K\rangle$$
$$\langle\langle M\ \mathsf{to}\ x.\, N,\ E\rangle \mid K\rangle \rightsquigarrow \langle\langle M,\ E\rangle \mid \mathsf{to}(x.\, N,\ E) :: K\rangle$$
$$\langle\langle \mathsf{return}\ V,\ E\rangle \mid \mathsf{to}(x.\, N,\ E') :: K\rangle \rightsquigarrow \langle\langle N,\ E'[x \mapsto (V,\ E)]\rangle \mid K\rangle$$
$$\langle\langle \mathsf{force}\ V,\ E\rangle \mid K\rangle \rightsquigarrow \langle\langle M',\ E'\rangle \mid K\rangle \quad \text{where } \langle M',\ E'\rangle = \mathsf{resolve}_U((V,\ E))$$
$$\langle\langle \mathsf{split}(V; x_1, x_2.\, M),\ E\rangle \mid K\rangle \rightsquigarrow \langle\langle M,\ E[x_1 \mapsto v_1][x_2 \mapsto v_2]\rangle \mid K\rangle \quad \text{where } (v_1, v_2) = \mathsf{resolve}_\times((V,\ E))$$
$$\langle\langle \mathsf{case}(V; x_1.\, M_1; x_2.\, M_2),\ E\rangle \mid K\rangle \rightsquigarrow \langle\langle M_i,\ E[x_i \mapsto v']\rangle \mid K\rangle \quad \text{where } \mathsf{inj}_i(v') = \mathsf{resolve}_+((V,\ E))$$

$\boxed{\Gamma \models E}$ **Environment types**

Env-Empty

$$\frac{}{\cdot \models \varnothing}$$

Env-Ext

$$\frac{\Gamma \models E \qquad \Gamma' \vdash^{\mathrm{v}} V_c' : A \qquad \Gamma' \models E'}{(\Gamma, x : A) \models E[x \mapsto (V_c',\ E')]}$$

$\boxed{\vdash \langle\langle M,\ E\rangle \mid K\rangle : \underline{D}}$ **Machine types**

Machine

$$\frac{\Gamma \vdash^{\mathrm{c}} M : \underline{C} \qquad \Gamma \models E \qquad \vdash K : \underline{C} \multimap \underline{D}}{\vdash \langle\langle M,\ E\rangle \mid K\rangle : \underline{D}}$$

$\boxed{\vdash K : \underline{C} \multimap \underline{D}}$ **Stack types**

Stk-Empty

$$\frac{}{\vdash \odot : \underline{C} \multimap \underline{C}}$$

Stk-Val

$$\frac{\Gamma \vdash^{\mathrm{v}} V : A \qquad \Gamma \models E \qquad \vdash K : \underline{C} \multimap \underline{D}}{\vdash (V,\ E) :: K : (A \to \underline{C}) \multimap \underline{D}}$$

Stk-Cont

$$\frac{\Gamma, x : A \vdash^{\mathrm{c}} N : \underline{C} \qquad \Gamma \models E \qquad \vdash K : \underline{C} \multimap \underline{D}}{\vdash \mathsf{to}(x.\, N,\ E) :: K : FA \multimap \underline{D}}$$

---

resolve **Resolution functions**

$$\mathsf{resolve}_U((\mathsf{thunk}\ M,\ E)) \stackrel{\mathrm{def}}{=} \langle M,\ E\rangle \qquad \mathsf{resolve}_\times(((V_1, V_2),\ E)) \stackrel{\mathrm{def}}{=} ((V_1,\ E),\ (V_2,\ E))$$
$$\mathsf{resolve}_+((\mathsf{inl}(V),\ E)) \stackrel{\mathrm{def}}{=} \mathsf{inj}_1((V,\ E)) \qquad \mathsf{resolve}_+((\mathsf{inr}(V),\ E)) \stackrel{\mathrm{def}}{=} \mathsf{inj}_2((V,\ E))$$

Fig. 7. The CEK machine for CBPV (`VClosure`, `_⊸_`, `_⇝_`).

A machine $\langle\langle M,\ E\rangle \mid K\rangle$ consists of a computation closure $\langle M,\ E\rangle$, which is the program it is currently evaluating, and a stack K. The stacks are similar to those of the CK machine, with the exception that they now carry closures rather than bare terms: a value frame $v :: K$ carries a value closure v, and a continuation frame $\mathsf{to}(x.\ N,\ E) :: K$ carries a computation N with one distinguished free variable x, and an environment providing values for its other free variables.

As with the CK machine, an application pushes its argument onto the stack, now as a value closure that captures the current environment; and a sequencing construct pushes a continuation, also capturing the current environment. The key difference lies in the rules that previously used substitutions. Instead of substituting, the CEK machine extends the environment with a new binding. For instance, when a λ-abstraction $\lambda x.\ M$ in environment E meets a value closure $(V',\ E')$ on the stack, the machine proceeds to evaluate the computation closure $\langle M,\ E[x \mapsto (V',\ E')]\rangle$, i.e. the body M of the function in the extended environment $E[x \mapsto (V',\ E')]$. Sequencing is similar: when a returner $\mathsf{return}\ V$ in environment E meets a continuation frame $\mathsf{to}(x.\ N,\ E') :: K$, the machine proceeds to install the computation closure $\langle N,\ E'[x \mapsto (V,\ E)]\rangle$, i.e. the body N of the continuation in its own environment, with the free x provided by the value closure $(V,\ E)$.

Closure Formation and the Resolution Functions. Recall that the CK machine only manipulates closed terms. Thus, every value it handles is headed by a constructor, and value eliminators can simply pattern-match on that. By contrast, the CEK machine evaluates computation closures, so a value V appearing in the program source may be either canonical or a variable. The machine must decide what to do in each case when forming a closure.

The convention is that the notation $(V,\ E)$ is a *smart constructor*. When V is a canonical value, $(V,\ E)$ is the literal pair of V and E. When V is a variable x, we define $(x,\ E)$ to be the lookup $E(x)$, which is itself a value closure. This means that variables are *dereferenced eagerly* when a closure is formed. This dereferencing is total because a well-typed environment binds every variable in scope. Consequently every value closure the machine produces is headed by a constructor.

On the other hand, each of the eliminators needs to inspect and deconstruct a value closure: $\mathsf{force}\ V$ must extract a computation closure from a value closure; $\mathsf{split}(V; x_1, x_2.\ M)$ must extract the two component value closures of a pair; and $\mathsf{case}(V; x_1.\ M_1; x_2.\ M_2)$ must select the branch to evaluate next. We encode this process as a family of *resolution functions*, one per value type constructor. Because the smart constructor guarantees that the input is constructor-headed, the resolution functions need only handle that case. For instance, when $\mathsf{force}\ V$ is computed, the closure $(V,\ E)$ is necessarily some $(\mathsf{thunk}\ M',\ E')$, and $\mathsf{resolve}_U((\mathsf{thunk}\ M',\ E')) \stackrel{\text{def}}{=} \langle M',\ E'\rangle$ delivers the next computation closure the machine should run; for $\mathsf{split}(V; x_1, x_2.\ M)$, the closure has the form $((V_1, V_2),\ E')$, and $\mathsf{resolve}_\times(((V_1, V_2),\ E')) \stackrel{\text{def}}{=} ((V_1,\ E'), (V_2,\ E'))$ delivers a value closure for each bound variable; the case of sums is analogous.

Canonicity. We prove canonicity for the CEK machine by the same biorthogonal method used for CK in Section 3 (`CEK.SyntacticLR.PureCanonicity`). The difference is that the logical relations trade in closures as their currency, and substitutions are replaced by environments.

The pole $\perp\!\!\!\perp$ is again the set of terminating states of the machine:

$$\perp\!\!\!\perp \stackrel{\text{def}}{=} \{S \mid S \rightsquigarrow^* \langle c_T \mid \odot\rangle \text{ for some terminal closure } c_T\}$$

We define four mutually dependent predicates $\mathcal{R}^v_A$, $\mathcal{R}^t_{\underline{C}}$, $\mathcal{R}^k_{\underline{C} \multimap \underline{D}}$, $\mathcal{R}^c_{\underline{C}}$ on value closures, terminal closures, stacks, and computation closures respectively. The defining clauses are given in Fig. 8.

The *value relation* is defined by induction on the value type, followed by case analysis on the value closure. The clauses mirror the CK case, checking that the constructor's arguments are also in the relation at the appropriate types. Note that the 'smart constructor' convention ensures that

$$\mathcal{R}^v_{\mathbf{1}}\ (\langle\rangle,\ E) \stackrel{\text{def}}{\equiv} \top \qquad \mathcal{R}^v_{A_1+A_2}\ (\mathsf{inl}(V),\ E) \stackrel{\text{def}}{\equiv} \mathcal{R}^v_{A_1}\ (V,\ E)$$

$$\mathcal{R}^v_{A_1+A_2}\ (\mathsf{inr}(V),\ E) \stackrel{\text{def}}{\equiv} \mathcal{R}^v_{A_2}\ (V,\ E) \qquad \mathcal{R}^v_{A_1\times A_2}\ ((V_1, V_2),\ E) \stackrel{\text{def}}{\equiv} \mathcal{R}^v_{A_1}\ (V_1,\ E) \wedge \mathcal{R}^v_{A_2}\ (V_2,\ E)$$

$$\mathcal{R}^v_{U\underline{C}}\ (\mathsf{thunk}\ M,\ E) \stackrel{\text{def}}{\equiv} \mathcal{R}^c_{\underline{C}}\ \langle M,\ E\rangle$$

$$\mathcal{R}^t_{FA}\ \langle \mathsf{return}\ V,\ E\rangle \stackrel{\text{def}}{\equiv} \mathcal{R}^v_A\ (V,\ E)$$

$$\mathcal{R}^t_{A\to\underline{C}}\ \langle \lambda x.\, M,\ E\rangle \stackrel{\text{def}}{\equiv} \forall v.\ \mathcal{R}^v_A\ v \Rightarrow \mathcal{R}^c_{\underline{C}}\ \langle M,\ E[x \mapsto v]\rangle$$

$$\mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K \stackrel{\text{def}}{\equiv} \forall c.\ \mathcal{R}^t_{\underline{C}}\ c \Rightarrow \langle c \mid K\rangle \in \perp\!\!\!\perp$$

$$\mathcal{R}^c_{\underline{C}}\ c \stackrel{\text{def}}{\equiv} \forall \underline{D}, K.\ \mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K \Rightarrow \langle c \mid K\rangle \in \perp\!\!\!\perp$$

Fig. 8. A syntactic logical relation for canonicity in the CEK machine ($\mathsf{R^v_\ni_}$, $\mathsf{R^t_\ni_,_}$, $\mathsf{R^k_\multimap_\ni_}$, $\mathsf{R^c_\ni_}$).

variables are dereferenced before being passed to the relation, so that the relation never has to consider the case of a free variable.

The *terminal closure relation* is defined by induction on the computation type, followed by case analysis on the terminal closure. For returners it defers to the value relation. For λ-abstractions $\langle \lambda x.\, M,\ E\rangle$ it requires that extending the environment with any value closure in the value relation yields a computation closure in the computation closure relation; this replaces the substitution used in the CK case. The *stack* and *computation closure relations* are defined via orthogonality, as in CK.

We may now show that

Lemma 6.1 (Stack constructors preserve the relation, `::-good`, `to::-good`).

(1) *If* $\mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K$ *and* $\mathcal{R}^v_A\ v$, *then* $\mathcal{R}^k_{(A\to\underline{C})\multimap\underline{D}}\ v :: K$.
(2) *If* $\mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K$ *and, for all* $\mathcal{R}^v_A\ v$, *we have* $\mathcal{R}^c_{\underline{C}}\ \langle N,\ E[x \mapsto v]\rangle$, *then* $\mathcal{R}^k_{FA\multimap\underline{D}}\ to(x.\, N,\ E) :: K$.

The proof is exactly as in Lemma 3.1, using the CEK transitions in place of the CK ones.

We write $\mathcal{R}^\Gamma\ E$ to mean that $\mathcal{R}^v_A\ E(x)$ for each $x : A \in \Gamma$. Where the CK logical relation closes open terms by substitution, this relation pairs them with an environment satisfying $\mathcal{R}^\Gamma\ E$.

The resolution functions interact well with the value relation:

Lemma 6.2 (Resolution, `U-resolve-R^v`, `×-resolve-R^v`, `+-resolve-R^v`).

(1) *If* $\mathcal{R}^v_{U\underline{C}}\ v$ *then* $\mathcal{R}^c_{\underline{C}}\ resolve_U(v)$.
(2) *If* $\mathcal{R}^v_{A_1\times A_2}\ v$ *and* $(v_1, v_2) = resolve_\times(v)$, *then* $\mathcal{R}^v_{A_1}\ v_1$ *and* $\mathcal{R}^v_{A_2}\ v_2$.
(3) *If* $\mathcal{R}^v_{A_1+A_2}\ v$ *then either* $resolve_+(v) = inj_1(v_1)$ *and* $\mathcal{R}^v_{A_1}\ v_1$, *or* $resolve_+(v) = inj_2(v_2)$ *and* $\mathcal{R}^v_{A_2}\ v_2$.

These hold by inversion on the value closure. We can now show the

Lemma 6.3 (Fundamental Lemma for CEK, `rel-val`, `rel-com`).

(1) *If* $\Gamma \vdash^v V : A$ *and* $\mathcal{R}^\Gamma\ E$ *then* $\mathcal{R}^v_A\ (V,\ E)$.
(2) *If* $\Gamma \vdash^c M : \underline{C}$ *and* $\mathcal{R}^\Gamma\ E$ *then* $\mathcal{R}^c_{\underline{C}}\ \langle M,\ E\rangle$.

Proof. By induction on typing derivations, using backward closure of the pole at each step. For illustration we show the case of application. Given $\Gamma \vdash^c M(V) : \underline{C}$ and $\mathcal{R}^\Gamma\ E$, we know $\mathcal{R}^c_{A\to\underline{C}}\ \langle M,\ E\rangle$ and $\mathcal{R}^v_A\ (V,\ E)$ by induction. For any $\mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K$, we form the stack $(V,\ E) :: K$,

which satisfies $\mathcal{R}^k_{(A\to\underline{C})\multimap\underline{D}}\ (V,\ E) :: K$ by Lemma 6.1(1). Therefore, as $\mathcal{R}^c_{A\to\underline{C}}\ \langle M,\ E\rangle$, we have $\langle\langle M,\ E\rangle \mid (V,\ E) :: K\rangle \in \perp\!\!\!\perp$. By backward closure of the pole, $\langle\langle M(V),\ E\rangle \mid K\rangle \in \perp\!\!\!\perp$. The cases for the value-type eliminators additionally invoke Lemma 6.2. □

As before, *we do not need to show the fundamental lemma for stacks.* We only need that the empty stack be in the relation:

Lemma 6.4 (`nil-good`). *For every computation type $\underline{C}$ we have $\mathcal{R}^k_{\underline{C}\multimap\underline{C}}\ \odot$.*

This holds because for *every* terminal closure c_T of type $\underline{C}$ we have $\langle c_T \mid \odot\rangle \in \perp\!\!\!\perp$.

From these facts we can deduce the

Theorem 6.5 (`canonicity`). *If $\vdash^c M : \underline{C}$, then $\langle\langle M,\ \varnothing\rangle \mid \odot\rangle \rightsquigarrow^* \langle c_T \mid \odot\rangle$ for some terminal closure c_T.*

Proof. By Lemma 6.3, $\mathcal{R}^c_{\underline{C}}\ \langle M,\ \varnothing\rangle$. Since $\mathcal{R}^k_{\underline{C}\multimap\underline{C}}\ \odot$, we get $\langle\langle M,\ \varnothing\rangle \mid \odot\rangle \in \perp\!\!\!\perp$. □

7 Environment Bimodels

Just as we abstracted the CK machine into bimodels (Section 4), we now abstract the CEK machine into *environment bimodels*. As the CEK machine computes with closures and environments rather than with substitution, environment bimodels replace substitutions by a *calculus of closures*.

Once again, Fig. 9 presents an environment bimodel $\mathfrak{B}$ in the same type-theoretic style. The sorts Ctx, VTy, and CTy are as before, and so are the families Val Γ A and Com Γ $\underline{C}$. The new family AVar Γ A represents *abstract variables*, and comes with an operation embed : AVar Γ $A \to$ Val Γ A that embeds them into values. The operations q and p construct abstract variables in the style of de Bruijn indexing, and lookup : AVar Γ $A \to$ Env $\Gamma \to$ VClos A retrieves the value closure bound to a variable in an environment.

There are also new families VClos A, Clos $\underline{C}$, Stk $\underline{C}$ $\underline{D}$, and Env Γ corresponding to value closures, computation closures, stacks, and environments. Each type former and constructor of CBPV has a semantic counterpart; these mirror the typing rules of Fig. 1 and the CEK transitions of Fig. 7. Finally, the theory has *resolution* operations, mirroring the CEK machine itself.

Coherence. Unlike their counterparts in the CK bimodel, the resolution functions act on closures rather than closed values, and so require some additional coherence.

As before, *constructor coherence* requires that resolving a closure formed from a constructor-headed value immediately yields the corresponding semantic constructor.

Additionally, *variable coherence* governs the interaction between abstract variables and the lookup operation. When a value closure is formed from a variable that indexes its environment, the result must agree with the closure that is referenced in the environment. This condition ensures that the indirection through variables coheres with closure formation.

Interpretation. Given an environment bimodel $\mathfrak{B}$, we define an interpretation of CBPV syntax into $\mathfrak{B}$ by induction on derivations, as for bimodels (`CEK.EnvBimodel.Interpretation`).

We then define a function $\mathsf{nf}(-)$ that expands a first-order value closure into a closed value. $\mathsf{nf}(-)$ recurses on the head constructor, re-closing each immediate subterm with the smart constructor:

$$\mathsf{nf}((\langle\rangle,\ E)) = \langle\rangle \qquad \mathsf{nf}((\mathsf{inl}(V),\ E)) = \mathsf{inl}(\mathsf{nf}((V,\ E)))$$
$$\mathsf{nf}((\mathsf{inr}(V),\ E)) = \mathsf{inr}(\mathsf{nf}((V,\ E))) \qquad \mathsf{nf}(((V_1, V_2),\ E)) = (\mathsf{nf}((V_1,\ E)), \mathsf{nf}((V_2,\ E)))$$

The coherence conditions then imply that the interpretation is injective on first-order value closures, up to normal form: value closures with equal denotation have equal normal forms.

Lemma 7.1 (Injectivity at first-order types, $[\![]\!]^{\mathsf{v}}$`-fo-inj`). *Let $\mathfrak{B}$ be an environment bimodel, A a first-order type, and v_1, v_2 value closures of type A. If $[\![v_1]\!]_{\mathfrak{B}} = [\![v_2]\!]_{\mathfrak{B}}$, then $nf(v_1) = nf(v_2)$.*

Sorts: $\mathsf{Ctx} : \mathsf{Set}_1$, $\mathsf{VTy} : \mathsf{Set}_1$, $\mathsf{CTy} : \mathsf{Set}_1$

Families:

$$\begin{aligned}
\mathsf{Val}\ \Gamma\ A &: \mathsf{Set}\\
\mathsf{AVar}\ \Gamma\ A &: \mathsf{Set}\\
\mathsf{Com}\ \Gamma\ \underline{C} &: \mathsf{Set}\\
\mathsf{VClos}\ A &: \mathsf{Set}\\
\mathsf{Clos}\ \underline{C} &: \mathsf{Set}\\
\mathsf{Stk}\ \underline{C}\ \underline{D} &: \mathsf{Set}\\
\mathsf{Env}\ \Gamma &: \mathsf{Set}
\end{aligned}$$

Closure formation:

$$\begin{aligned}
\mathsf{vclose} &: \mathsf{Val}\ \Gamma\ A \to \mathsf{Env}\ \Gamma \to \mathsf{VClos}\ A\\
\mathsf{close} &: \mathsf{Com}\ \Gamma\ \underline{C} \to \mathsf{Env}\ \Gamma \to \mathsf{Clos}\ \underline{C}
\end{aligned}$$

Environment formation:

$$\begin{aligned}
\varnothing &: \mathsf{Env}\ \emptyset\\
-[-] &: \mathsf{Env}\ \Gamma \to \mathsf{VClos}\ A \to \mathsf{Env}\ \Gamma.A
\end{aligned}$$

Abstract variable operations:

$$\begin{aligned}
\mathsf{embed} &: \mathsf{AVar}\ \Gamma\ A \to \mathsf{Val}\ \Gamma\ A\\
\mathsf{q} &: \mathsf{AVar}\ \Gamma.A\ A\\
\mathsf{p} &: \mathsf{AVar}\ \Gamma\ B \to \mathsf{AVar}\ \Gamma.A\ B\\
\mathsf{lookup} &: \mathsf{AVar}\ \Gamma\ A \to \mathsf{Env}\ \Gamma \to \mathsf{VClos}\ A
\end{aligned}$$

subject to the equations

$$\begin{aligned}
\mathsf{lookup}(\mathsf{q},\ E[v]) &= v\\
\mathsf{lookup}(\mathsf{p}(x),\ E[v]) &= \mathsf{lookup}(x,\ E)
\end{aligned}$$

Resolution functions:

$$\begin{aligned}
\mathsf{resolve}_U(-) &: \mathsf{VClos}\ (U\underline{C}) \to \mathsf{Clos}\ \underline{C}\\
\mathsf{resolve}_\times(-) &: \mathsf{VClos}\ (A_1 \times A_2)\\
&\qquad \to \mathsf{VClos}\ A_1 \times \mathsf{VClos}\ A_2\\
\mathsf{resolve}_+(-) &: \mathsf{VClos}\ (A_1 + A_2)\\
&\qquad \to \mathsf{VClos}\ A_1 + \mathsf{VClos}\ A_2
\end{aligned}$$

subject to coherence equations (see below).

Type and context operations: As for CK bimodels (Fig. 4).

Term constructors: For each typing rule in Fig. 1, a corresponding operation.

Stack constructors: For each stack typing rule in Fig. 7, a corresponding operation on $\mathsf{Stk}\ \underline{C}\ \underline{D}$.

Constructor coherence: Resolution on value closures agrees with constructors; e.g.

$$\mathsf{resolve}_+(\mathsf{vclose}(\mathsf{inl}(V),\ E)) = \mathsf{inj}_1(\mathsf{vclose}(V, E))$$

and similarly for $\mathsf{inr}(V)$, pairs, and thunks.

Variable coherence: Constructing the closure of a variable agrees with looking it up:

$$\mathsf{vclose}(\mathsf{embed}(x),\ E) = \mathsf{lookup}(x,\ E)$$

Machine states and transition: A relation

$$(- \rightsquigarrow -) : \mathsf{State}\ \underline{C} \to \mathsf{State}\ \underline{C} \to \mathsf{Set}$$

where $\mathsf{State}\ \underline{C} \stackrel{\text{def}}{=} (\underline{D} : \mathsf{CTy}) \times \mathsf{Clos}\ \underline{D} \times \mathsf{Stk}\ \underline{D}\ \underline{C}$, closed under the rules of Fig. 7, stated in terms of the environment bimodel operations, e.g.

$$\langle \mathsf{close}(M(V), E) \mid K \rangle \rightsquigarrow \langle \mathsf{close}(M, E) \mid \mathsf{vclose}(V, E) :: K \rangle.$$

Fig. 9. An environment bimodel (`EnvBimodel`).

7.1 The Syntactic Environment Bimodel

As with the CK machine, the syntax of the CEK machine itself forms an environment bimodel. The families $\mathsf{Val}\ \Gamma\ A$ and $\mathsf{Com}\ \Gamma\ \underline{C}$ are the syntactic typing judgements; $\mathsf{Clos}\ \underline{C}$, $\mathsf{Stk}\ \underline{C}\ \underline{D}$, and $\mathsf{Env}\ \Gamma$ are the syntactic notions of computation closure, stack, and environment. As in the CEK machine, value closures $\mathsf{VClos}\ A$ are restricted to pair an environment with a constructor-headed value, and vclose is implemented as the smart constructor of the CEK machine. Computation closure formation is direct pairing, $\mathsf{close}(M,\ E) \stackrel{\text{def}}{=} \langle M,\ E \rangle$. The resolution functions are the syntactic resolution functions from Fig. 7, and all coherence conditions hold (`CEK.EnvBimodel.SyntacticEnvBimodel`).

As with the CK case, the interpretation into the syntactic environment bimodel is the identity: types, terms, closures, environments, and stacks are all mapped to themselves.

7.2 The Set-Theoretic Environment Bimodel

The set-theoretic environment bimodel gives a denotational semantics for the CEK machine. As before, types are interpreted as sets. Values and computations are functions from contexts, and stacks are functions:

$$\mathsf{Val}\ \Gamma\ A = \Gamma \to A \qquad \mathsf{Com}\ \Gamma\ C = \Gamma \to C \qquad \mathsf{Stk}\ C\ D = C \to D$$

Value closures, computation closures, and environments are simply identities:

$$\mathsf{VClos}\ A = A \qquad \mathsf{Clos}\ C = C \qquad \mathsf{Env}\ \Gamma = \Gamma$$

A value closure is simply an element of A: closing a function $V : \Gamma \to A$ with an environment $\gamma \in \Gamma$ yields $V(\gamma) \in A$. Thus, denotationally, closure formation is simply function application. Environments are elements of the context, and environment extension is pairing.

Finally, abstract variables coincide with values, i.e. $\mathsf{AVar}\ \Gamma\ A \stackrel{\text{def}}{=} \Gamma \to A$. Thus, embed is the identity, q is the second projection $\Gamma \times A \to A$, p precomposes with the first projection, and $\mathsf{lookup}(x, \gamma) = x(\gamma)$. Variable coherence then holds: $\mathsf{vclose}(\mathsf{embed}(x), \gamma) = \mathsf{lookup}(x, \gamma)$, as both unfold to $x(\gamma)$ (`CEK.EnvBimodel.TyEnvBimodel`).

The resolution functions are all the identity. For example, $\mathsf{resolve}_+(v) = v$, since a closed value $v \in A_1 + A_2$ is already either $\mathsf{inj}_1(v_1)$ or $\mathsf{inj}_2(v_2)$. At the denotational level, there is no indirection through variables to resolve. The transition relation identifies states with the same denotation, and all β-rules hold, as in the set-theoretic bimodel of Section 4.2.

7.3 Biorthogonal Logical Relations for Environment Bimodels

We now define logical relations parametric in the choice of environment bimodel, paralleling the bimodel development of Section 4.3.

Unary Logical Relations. Fix an environment bimodel $\mathfrak{B}$, and let $\perp\!\!\!\perp$ be a pole on closed states, closed under inverse reduction. The unary relation is defined by induction on value types for closures, and by orthogonality for stacks and computation closures (`CEK.SemanticLR.Unary`). The defining clauses are given in Fig. 10.

We inductively define a predicate $\mathcal{R}^\Gamma$: Env $[\![\Gamma]\!] \to$ Set on environments (R^e) by the rules

$$\frac{}{\mathcal{R}^\emptyset\ \varnothing} \qquad \frac{\mathcal{R}^\Gamma\ E \qquad \mathcal{R}^v_A\ v}{\mathcal{R}^{\Gamma \cdot A}\ E[v]}$$

Lemma 7.2 (Fundamental lemma, unary, `rel-val`, `rel-com`).
Let $\mathfrak{B}$ be an environment bimodel and $\perp\!\!\!\perp$ a pole.

(1) *If $\Gamma \vdash^v V : A$ and $\mathcal{R}^\Gamma\ E$, then $\mathcal{R}^v_A\ \mathit{vclose}^{\mathfrak{B}}([\![V]\!], E)$.*
(2) *If $\Gamma \vdash^c M : \underline{C}$ and $\mathcal{R}^\Gamma\ E$, then $\mathcal{R}^c_{\underline{C}}\ \mathit{close}^{\mathfrak{B}}([\![M]\!], E)$.*

Binary Logical Relations. Let $\mathfrak{B}_1$ and $\mathfrak{B}_2$ be environment bimodels with respective interpretations $[\![-]\!]_{\mathfrak{B}_1}$ and $[\![-]\!]_{\mathfrak{B}_2}$, and let $\perp\!\!\!\perp$ be a binary pole on pairs of closed states (`CEK.SemanticLR.Binary`). The binary logical relation clauses are given in Fig. 11. We inductively define a binary relation $\mathcal{R}^\Gamma$: Env $[\![\Gamma]\!]_{\mathfrak{B}_1} \to$ Env $[\![\Gamma]\!]_{\mathfrak{B}_2} \to$ Set on pairs of environments by the rules

$$\frac{}{\varnothing_1\ \mathcal{R}^\emptyset\ \varnothing_2} \qquad \frac{E_1\ \mathcal{R}^\Gamma\ E_2 \qquad v_1\ \mathcal{R}^v_A\ v_2}{E_1[v_1]\ \mathcal{R}^{\Gamma \cdot A}\ E_2[v_2]}$$

$$\mathcal{R}^v_{\mathbf{1}}\ v \stackrel{\text{def}}{\equiv} \top$$

$$\mathcal{R}^v_{A_1+A_2}\ v \stackrel{\text{def}}{\equiv} (\exists v_1 : \mathsf{VClos}\ [\![A_1]\!].\ \mathsf{resolve}_+(v) = \mathsf{inj}_1(v_1)\ \wedge\ \mathcal{R}^v_{A_1}\ v_1) \vee (\exists v_2 : \mathsf{VClos}\ [\![A_2]\!].\ \mathsf{resolve}_+(v) = \mathsf{inj}_2(v_2)\ \wedge\ \mathcal{R}^v_{A_2}\ v_2)$$

$$\mathcal{R}^v_{A_1\times A_2}\ v \stackrel{\text{def}}{\equiv} \exists v_1 : \mathsf{VClos}\ [\![A_1]\!],\ v_2 : \mathsf{VClos}\ [\![A_2]\!].\ \mathsf{resolve}_\times(v) = (v_1, v_2)\ \wedge\ \mathcal{R}^v_{A_1}\ v_1\ \wedge\ \mathcal{R}^v_{A_2}\ v_2$$

$$\mathcal{R}^v_{U\underline{C}}\ v \stackrel{\text{def}}{\equiv} \mathcal{R}^c_{\underline{C}}\ \mathsf{resolve}_U(v)$$

$$\mathcal{R}^t_{FA}\ c \stackrel{\text{def}}{\equiv} \exists \Gamma : \mathsf{Ctx},\ V : \mathsf{Val}\ \Gamma\ [\![A]\!],\ E : \mathsf{Env}\ \Gamma.\ c = \mathsf{close}^{\mathfrak{B}}(\mathsf{return}^{\mathfrak{B}}(V),\ E)\ \wedge\ \mathcal{R}^v_A\ \mathsf{vclose}^{\mathfrak{B}}(V, E)$$

$$\mathcal{R}^t_{A\to\underline{C}}\ c \stackrel{\text{def}}{\equiv} \exists \Gamma : \mathsf{Ctx},\ M : \mathsf{Com}\ \Gamma\centerdot[\![A]\!]\ [\![\underline{C}]\!],\ E : \mathsf{Env}\ \Gamma.\ c = \mathsf{close}^{\mathfrak{B}}(\lambda^{\mathfrak{B}}(M),\ E)\ \wedge\ \forall v.\ \mathcal{R}^v_A\ v \Rightarrow \mathcal{R}^c_{\underline{C}}\ \mathsf{close}^{\mathfrak{B}}(M,\ E[v])$$

$$\mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K \stackrel{\text{def}}{\equiv} \forall c : \mathsf{Clos}\ \underline{C}.\ \mathcal{R}^t_{\underline{C}}\ c\ \Rightarrow\ \langle c \mid K\rangle \in \perp\!\!\!\perp$$

$$\mathcal{R}^c_{\underline{C}}\ c \stackrel{\text{def}}{\equiv} \forall \underline{D},\ K : \mathsf{Stk}\ [\![\underline{C}]\!]\ [\![\underline{D}]\!].\ \mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K \Rightarrow \langle c \mid K\rangle \in \perp\!\!\!\perp$$

Fig. 10. Unary logical relation for environment bimodels (R^{v}, R^{t}, R^{k}, R^{c}).

Lemma 7.3 (Fundamental lemma, binary, `rel-val`, `rel-com`).
Let $\mathfrak{B}_1, \mathfrak{B}_2$ be environment bimodels and $\perp\!\!\!\perp$ a binary pole.

(1) *If* $\Gamma \vdash^{\mathsf{v}} V : A$ *and* $E_1\ \mathcal{R}^\Gamma\ E_2$ *then* $\mathit{vclose}_1([\![V]\!]_{\mathfrak{B}_1}, E_1)\ \mathcal{R}^v_A\ \mathit{vclose}_2([\![V]\!]_{\mathfrak{B}_2}, E_2)$.

(2) *If* $\Gamma \vdash^{c} M : \underline{C}$ *and* $E_1\ \mathcal{R}^\Gamma\ E_2$ *then* $\mathit{close}_1([\![M]\!]_{\mathfrak{B}_1}, E_1)\ \mathcal{R}^c_{\underline{C}}\ \mathit{close}_2([\![M]\!]_{\mathfrak{B}_2}, E_2)$.

8 Applications of Environment Bimodels

The logical relations of Section 7 are generic in the choice of environment bimodel and pole, just as in the CK case. We now show that instantiating them yields the CEK analogues of the results in Section 5, plus a new result: a formal simulation of the CK machine by the CEK machine.

A unary logical relation on the syntactic environment bimodel yields **canonicity** for the CEK machine (Section 8.1). A binary logical relation between the syntactic and set-theoretic environment bimodels yields **adequacy** of the set-theoretic model for the CEK machine (Section 8.2). Finally, we show how to construct an environment bimodel from the pieces of the CK machine. This leads to a **simulation theorem**, or **adequacy of the CK machine for the CEK machine** (Section 8.3).

8.1 Canonicity

Consider the following unary pole on the syntactic environment bimodel:

$$\begin{array}{lcl} \perp\!\!\!\perp_{\mathsf{Can}} & : & (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Syn}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Syn}}} \to \mathsf{Set} \\ \perp\!\!\!\perp_{\mathsf{Can}}\ (FA)\ S & \stackrel{\text{def}}{=} & S \rightsquigarrow^* \langle\langle \mathsf{return}\ V,\ E\rangle \mid \odot\rangle \text{ for some } V, E \\ \perp\!\!\!\perp_{\mathsf{Can}}\ (A \to \underline{C})\ _ & \stackrel{\text{def}}{=} & \top \end{array}$$

A CEK state of type FA is in the pole if the machine eventually reaches a returner closure with an empty stack. States of function type are vacuously in the pole; the choice here is immaterial,

$$
\begin{aligned}
v_1\ \mathcal{R}^v_1\ v_2 &\stackrel{\text{def}}{\equiv} \top \\
v_1\ \mathcal{R}^v_{A_1+A_2}\ v_2 &\stackrel{\text{def}}{\equiv} (\exists v_1' : \mathsf{VClos}\ [\![A_1]\!].\ \exists v_2' : \mathsf{VClos}\ [\![A_1]\!].\\
&\qquad \mathsf{resolve}_{+,1}(v_1) = \mathsf{inj}_1(v_1') \wedge \mathsf{resolve}_{+,2}(v_2) = \mathsf{inj}_1(v_2')\ \wedge\ v_1'\ \mathcal{R}^v_{A_1}\ v_2') \vee \\
&\quad (\exists v_1' : \mathsf{VClos}\ [\![A_2]\!].\ \exists v_2' : \mathsf{VClos}\ [\![A_2]\!].\\
&\qquad \mathsf{resolve}_{+,1}(v_1) = \mathsf{inj}_2(v_1') \wedge \mathsf{resolve}_{+,2}(v_2) = \mathsf{inj}_2(v_2')\ \wedge\ v_1'\ \mathcal{R}^v_{A_2}\ v_2') \\
v_1\ \mathcal{R}^v_{A_1\times A_2}\ v_2 &\stackrel{\text{def}}{\equiv} \exists w_1 : \mathsf{VClos}\ [\![A_1]\!],\ w_1' : \mathsf{VClos}\ [\![A_2]\!],\ w_2 : \mathsf{VClos}\ [\![A_1]\!],\ w_2' : \mathsf{VClos}\ [\![A_2]\!].\\
&\qquad \mathsf{resolve}_{\times,1}(v_1) = (w_1, w_1') \wedge \mathsf{resolve}_{\times,2}(v_2) = (w_2, w_2')\ \wedge\ w_1\ \mathcal{R}^v_{A_1}\ w_2\ \wedge\ w_1'\ \mathcal{R}^v_{A_2}\ w_2' \\
v_1\ \mathcal{R}^v_{U\underline{C}}\ v_2 &\stackrel{\text{def}}{\equiv} \mathsf{resolve}_{U,1}(v_1)\ \mathcal{R}^c_{\underline{C}}\ \mathsf{resolve}_{U,2}(v_2) \\
c_1\ \mathcal{R}^t_{FA}\ c_2 &\stackrel{\text{def}}{\equiv} \exists \Gamma_1, \Gamma_2 : \mathsf{Ctx},\ V_1 : \mathsf{Val}\ \Gamma_1\ [\![A]\!],\ E_1 : \mathsf{Env}\ \Gamma_1,\ V_2 : \mathsf{Val}\ \Gamma_2\ [\![A]\!],\ E_2 : \mathsf{Env}\ \Gamma_2.\\
&\qquad c_1 = \mathsf{close}_1(\mathsf{return}^{\mathfrak{B}_1}(V_1),\ E_1) \wedge c_2 = \mathsf{close}_2(\mathsf{return}^{\mathfrak{B}_2}(V_2),\ E_2)\ \wedge \\
&\qquad \mathsf{vclose}_1(V_1, E_1)\ \mathcal{R}^v_A\ \mathsf{vclose}_2(V_2, E_2) \\
c_1\ \mathcal{R}^t_{A\to\underline{C}}\ c_2 &\stackrel{\text{def}}{\equiv} \exists \Gamma_1, \Gamma_2 : \mathsf{Ctx},\ M_1 : \mathsf{Com}\ \Gamma_1.[\![A]\!]\ [\![\underline{C}]\!],\ E_1 : \mathsf{Env}\ \Gamma_1,\ M_2 : \mathsf{Com}\ \Gamma_2.[\![A]\!]\ [\![\underline{C}]\!],\ E_2 : \mathsf{Env}\ \Gamma_2.\\
&\qquad c_1 = \mathsf{close}_1(\lambda^{\mathfrak{B}_1}(M_1),\ E_1) \wedge c_2 = \mathsf{close}_2(\lambda^{\mathfrak{B}_2}(M_2),\ E_2)\ \wedge \\
&\qquad \forall v_1, v_2.\ v_1\ \mathcal{R}^v_A\ v_2 \Rightarrow \mathsf{close}_1(M_1,\ E_1[v_1])\ \mathcal{R}^c_{\underline{C}}\ \mathsf{close}_2(M_2,\ E_2[v_2]) \\
K_1\ \mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K_2 &\stackrel{\text{def}}{\equiv} \forall c_1, c_2.\ c_1\ \mathcal{R}^t_{\underline{C}}\ c_2\ \Rightarrow\ (\langle c_1 \mid K_1\rangle, \langle c_2 \mid K_2\rangle) \in \perp\!\!\!\perp \\
c_1\ \mathcal{R}^c_{\underline{C}}\ c_2 &\stackrel{\text{def}}{\equiv} \forall \underline{D}.\ \forall K_1 : \mathsf{Stk}\ [\![\underline{C}]\!]\ [\![\underline{D}]\!],\ K_2 : \mathsf{Stk}\ [\![\underline{C}]\!]\ [\![\underline{D}]\!].\ K_1\ \mathcal{R}^k_{\underline{C}\multimap\underline{D}}\ K_2 \Rightarrow (\langle c_1 \mid K_1\rangle, \langle c_2 \mid K_2\rangle) \in \perp\!\!\!\perp
\end{aligned}
$$

Fig. 11. Binary logical relation for environment bimodels (CEK) (R^{v}, R^{t}, R^{k}, R^{c}).

as canonicity concerns only returner types. $\perp\!\!\!\perp_{\mathsf{Can}}$ is evidently closed under inverse evaluation. We therefore obtain unary logical relations. By Lemma 7.2 we know that $\mathcal{R}^v_A\ \mathsf{vclose}^{\mathfrak{B}}([\![V]\!], E)$ and $\mathcal{R}^c_{\underline{C}}\ \mathsf{close}^{\mathfrak{B}}([\![M]\!], E)$ for all $\vdash^{\mathsf{v}} V : A$ and $\vdash^{\mathsf{c}} M : \underline{C}$, whenever $\mathcal{R}^\Gamma\ E$ (`CEK.SemanticLR.Canonicity`).

We additionally show that $\mathcal{R}^k_{FA\multimap FA}\ \odot$ for all value types A. This requires showing that for every terminal closure c with $\mathcal{R}^t_{FA}\ c$, the state $\langle c \mid \odot\rangle$ is in the pole. But $\mathcal{R}^t_{FA}\ c$ in the syntactic environment bimodel implies that c is a returner closure, so $\langle c \mid \odot\rangle$ is trivially in the pole.

Putting $\mathcal{R}^c_{FA}\ \mathsf{close}([\![M]\!], \varnothing)$ together with $\mathcal{R}^k_{FA\multimap FA}\ \odot$, we obtain:

Theorem 8.1 (`canonicity`). *If $\vdash^c M : FA$, then $\langle\langle M,\ \varnothing\rangle \mid \odot\rangle \rightsquigarrow^* \langle\langle \mathsf{return}\ V,\ E\rangle \mid \odot\rangle$ for some value V and environment E.*

8.2 Adequacy

We prove that the set-theoretic environment bimodel $\mathfrak{B}_{\mathsf{Set}}$ is *adequate* with respect to the syntactic environment bimodel $\mathfrak{B}_{\mathsf{Syn}}$ (`CEK.SemanticLR.Adequacy`). As in the CK case, adequacy is restricted to first-order types, with the observable content of a value closure taken to be its normal form.

We define the following binary pole between $\mathfrak{B}_{\mathsf{Set}}$ and $\mathfrak{B}_{\mathsf{Syn}}$:

$$
\begin{array}{lcl}
\perp\!\!\!\perp_{\mathsf{Adeq}} & : & (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Set}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Set}}} \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Syn}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Syn}}} \to \mathsf{Set} \\
\perp\!\!\!\perp_{\mathsf{Adeq}}\ (FA)\ S_1\ S_2 & \stackrel{\mathrm{def}}{=} & \mathsf{FO}(A) \to \forall V, E.\ S_1 \rightsquigarrow^* [\![\langle\langle \mathsf{return}\ V,\ E\rangle \mid \odot\rangle]\!]_{\mathfrak{B}_{\mathsf{Set}}} \to \\
& & \exists V', E'.\ S_2 \rightsquigarrow^* \langle\langle \mathsf{return}\ V',\ E'\rangle \mid \odot\rangle \ \wedge\ \mathsf{nf}((V,\ E)) = \mathsf{nf}((V',\ E')) \\
\perp\!\!\!\perp_{\mathsf{Adeq}}\ _\ _\ _ & \stackrel{\mathrm{def}}{=} & \bot
\end{array}
$$

The pole says: if the set-theoretic model predicts that the computation returns a value closure $(V,\ E)$ at a first-order type, then the CEK machine must also terminate at a returner closure, potentially a different one, but with the same normal form. We instantiate the binary logical relation between $\mathfrak{B}_{\mathsf{Set}}$ and $\mathfrak{B}_{\mathsf{Syn}}$, and obtain the following analogue of the graph lemma (Lemma 5.2).

Lemma 8.2 (Graph Lemma for CEK, $\mathsf{R^v}$-sound). *If $v_1\ \mathcal{R}^v_\sigma\ v_2$ then $v_1 = [\![v_2]\!]_{\mathfrak{B}_{Set}}$.*

Using this we can show $\odot\ \mathcal{R}^k_{FA \multimap FA}\ \odot$ for any A. By definition of $\mathcal{R}^k_{FA \multimap FA}$ we must check that, whenever $c_1\ \mathcal{R}^t_{FA}\ c_2$, the pair $((c_1, \odot), (c_2, \odot))$ lies in $\perp\!\!\!\perp_{\mathsf{Adeq}}$. Unfolding $\mathcal{R}^t_{FA}$ exhibits c_1 and c_2 as returner closures whose underlying value closures are related, say $v_1\ \mathcal{R}^v_A\ v_2$. Then the premises of the pole unfold to $\mathsf{FO}(A)$ and $v_1 = [\![(V,\ E)]\!]_{\mathfrak{B}_{\mathsf{Set}}}$ for some V and E. Now Lemma 8.2 yields $[\![(V,\ E)]\!]_{\mathfrak{B}_{\mathsf{Set}}} = v_1 = [\![v_2]\!]_{\mathfrak{B}_{\mathsf{Set}}}$, and Lemma 7.1 then gives $\mathsf{nf}((V,\ E)) = \mathsf{nf}(v_2)$. We obtain the

Theorem 8.3 (adequacy). *Let σ be a first-order type, $\vdash^c M : F\sigma$, and let $(V,\ E)$ be a value closure of type σ. If $[\![\langle M,\ \varnothing\rangle]\!]_{\mathfrak{B}_{Set}} = [\![\langle \mathsf{return}\ V,\ E\rangle]\!]_{\mathfrak{B}_{Set}}$ then $\langle\langle M,\ \varnothing\rangle \mid \odot\rangle \rightsquigarrow^* \langle\langle \mathsf{return}\ V',\ E'\rangle \mid \odot\rangle$ for some V' and E' with $nf((V,\ E)) = nf((V',\ E'))$.*

Proof. By Lemma 7.3 applied to $\mathfrak{B}_{\mathsf{Set}}$ and $\mathfrak{B}_{\mathsf{Syn}}$ we have

$$\mathsf{close}_{\mathfrak{B}_{\mathsf{Set}}}([\![M]\!]_{\mathfrak{B}_{\mathsf{Set}}}, \varnothing)\ \mathcal{R}^c_{F\sigma}\ \mathsf{close}_{\mathfrak{B}_{\mathsf{Syn}}}([\![M]\!]_{\mathfrak{B}_{\mathsf{Syn}}}, \varnothing).$$

Pairing this with $\odot\ \mathcal{R}^k_{F\sigma \multimap F\sigma}\ \odot$ loads both closures onto the empty stack and yields $(S_1, S_2) \in \perp\!\!\!\perp_{\mathsf{Adeq}}$, where $S_1 = (\mathsf{close}_{\mathfrak{B}_{\mathsf{Set}}}([\![M]\!]_{\mathfrak{B}_{\mathsf{Set}}}, \varnothing), \odot)$ and $S_2 = (\mathsf{close}_{\mathfrak{B}_{\mathsf{Syn}}}([\![M]\!]_{\mathfrak{B}_{\mathsf{Syn}}}, \varnothing), \odot) = \langle\langle M,\ \varnothing\rangle \mid \odot\rangle$. Using V, E, the fact that σ is first-order, and the hypothesis (on $\mathfrak{B}_{\mathsf{Set}}$ a step is just an equation), the pole gives V', E', $\langle\langle M,\ \varnothing\rangle \mid \odot\rangle \rightsquigarrow^* \langle\langle \mathsf{return}\ V',\ E'\rangle \mid \odot\rangle$, and the matching normal forms. □

8.3 Adequacy of CK for CEK

Somewhat surprisingly, the CK machine itself can be packaged as an environment bimodel $\mathfrak{B}_{\mathsf{CK}}$ (`Bridges.CKEnvBimodel`). Instantiating the binary logical relation with $\mathfrak{B}_{\mathsf{CK}}$ and the syntactic environment bimodel $\mathfrak{B}_{\mathsf{Syn}}$ of the CEK machine then proves *adequacy of the CK machine for the CEK machine* (`Bridges.CK-CEK`). Explicitly: fix a first-order value type σ; whenever the CK machine evaluates a computation of type $F\sigma$ to $\mathsf{return}\ V$, the CEK machine evaluates it to a returner closure $\langle \mathsf{return}\ V',\ E'\rangle$ whose corresponding value closure $(V',\ E')$ has the same normal form as V, that is, $\mathsf{nf}((V,\ \varnothing)) = \mathsf{nf}((V',\ E'))$. Thus, at first-order types, every CK result is matched by a CEK result.

We define $\mathfrak{B}_{\mathsf{CK}}$ by taking environments to be closing substitutions and closures to be closed terms. Concretely: value closures $\mathsf{VClos}\ A$ are closed values $\vdash^{\mathsf{v}} V : A$; computation closures $\mathsf{Clos}\ \underline{C}$ are closed computations $\vdash^{\mathsf{c}} M : \underline{C}$; and environments $\mathsf{Env}\ \Gamma$ are closing substitutions $\mathsf{Sub}\ \varnothing\ \Gamma$. Closure formation applies the substitution, i.e. $\mathsf{vclose}(V, \gamma) = V[\gamma]^{\mathsf{v}}$ and $\mathsf{close}(M, \gamma) = M[\gamma]^{\mathsf{c}}$. Finally, stacks $\mathsf{Stk}\ \underline{C}\ \underline{D}$ are CK stacks $\mathsf{Stk}\ \varnothing\ \underline{C}\ \underline{D}$, and transitions are CK machine transitions.

We define a binary pole between $\mathfrak{B}_{\mathsf{CK}}$ and $\mathfrak{B}_{\mathsf{Syn}}$:

$$
\begin{array}{lcl}
\perp\!\!\!\perp_{\mathsf{Sim}} & : & (\underline{C} : \mathsf{CompTy}) \to \mathsf{State}^{\mathfrak{B}_{\mathsf{CK}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{CK}}} \to \mathsf{State}^{\mathfrak{B}_{\mathsf{Syn}}}\ [\![\underline{C}]\!]_{\mathfrak{B}_{\mathsf{Syn}}} \to \mathsf{Set} \\
\perp\!\!\!\perp_{\mathsf{Sim}}\ (FA)\ S_1\ S_2 & \stackrel{\mathrm{def}}{=} & \mathsf{FO}(A) \to \forall V.\ S_1 \rightsquigarrow^* \langle \mathsf{return}\ V \mid \odot\rangle \to \\
& & \exists V', E'.\ S_2 \rightsquigarrow^* \langle\langle \mathsf{return}\ V',\ E'\rangle \mid \odot\rangle \ \wedge\ \mathsf{nf}((V,\ \varnothing)) = \mathsf{nf}((V',\ E')) \\
\perp\!\!\!\perp_{\mathsf{Sim}}\ (A \to \underline{C})\ _\ _ & \stackrel{\mathrm{def}}{=} & \bot
\end{array}
$$

The pole says: if the CK machine terminates at a first-order return value return V, then the CEK machine also terminates at a returner closure with the same first-order normal form. Backward closure on the CK side relies on *determinism* of the CK machine, used to transfer the return observation across inverse CK steps.

The main technical ingredient is the following:

Lemma 8.4 (Related values have equal normal forms, $\mathsf{R^v\text{-}nf}$). *If $V\ \mathcal{R}^v_\sigma\ v$ where V is a closed value (from $\mathfrak{B}_{CK}$) and v is a value closure (from $\mathfrak{B}_{Syn}$), then $nf((V, \varnothing)) = nf(v)$.*

This is proved by induction on first-order types. We can use it to show $\odot\ \mathcal{R}^k_{FA\multimap FA}\ \odot$, which yields the following theorem.

Theorem 8.5 (CK to CEK, $\mathsf{ck\text{-}cek}$). *Let σ be a first-order value type and $\vdash^c M : F\sigma$. If*

$$\langle M \mid \odot \rangle \rightsquigarrow^* \langle \mathit{return}\ V \mid \odot \rangle,$$

then $\langle\langle M, \varnothing\rangle \mid \odot\rangle \rightsquigarrow^* \langle\langle \mathit{return}\ V', E'\rangle \mid \odot\rangle$ for some V' and E' with $nf((V, \varnothing)) = nf((V', E'))$.

Proof. By Lemma 7.3 applied to $\mathfrak{B}_{\mathsf{CK}}$ and $\mathfrak{B}_{\mathsf{Syn}}$, we have

$$\mathsf{close}_{\mathfrak{B}_{\mathsf{CK}}}(\llbracket M \rrbracket_{\mathfrak{B}_{\mathsf{CK}}}, \epsilon)\ \mathcal{R}^c_{F\sigma}\ \mathsf{close}_{\mathfrak{B}_{\mathsf{Syn}}}(\llbracket M \rrbracket_{\mathfrak{B}_{\mathsf{Syn}}}, \varnothing).$$

Pairing this with $\odot\ \mathcal{R}^k_{F\sigma\multimap F\sigma}\ \odot$ yields $(S_1, S_2) \in \perp\!\!\!\perp_{\mathsf{Sim}}$, where $S_1 = \langle M \mid \odot\rangle$ and $S_2 = \langle\langle M, \varnothing\rangle \mid \odot\rangle$. Both equalities hold because interpretation is the identity in each bimodel; on the CK side, closure formation additionally applies the empty closing substitution, which leaves the closed term M unchanged. Unfolding $\perp\!\!\!\perp_{\mathsf{Sim}}$ at $\mathsf{FO}(\sigma)$ and V, the CK hypothesis discharges its antecedent $S_1 \rightsquigarrow^* \langle \mathsf{return}\ V \mid \odot\rangle$, and the pole then returns the witnesses V', E', the CEK reduction $\langle\langle M, \varnothing\rangle \mid \odot\rangle \rightsquigarrow^* \langle\langle \mathsf{return}\ V', E'\rangle \mid \odot\rangle$, and the matching normal form. □

9 Related Work

This paper owes an enormous intellectual debt to a diverse body of work on operational semantics, denotational semantics, and logical relations.

Biorthogonality. The origins of biorthogonality are lost in the mists of time. The earliest known occurrence is in Girard's phase semantics for linear logic [1987]. It was rediscovered by Pitts [1998] for making logical relations for existential types, and extended to more advanced language features, e.g. local state [Pitts and Stark 1998], monadic types [Lindley and Stark 2005], and parametric polymorphism [Pitts 2000]; see Pitts [2010] for a tutorial. It was also rediscovered by Krivine [2009, 2001] in the context of classical realizability. Melliès and Vouillon [2005; 2004] use biorthogonality to construct a model with recursive types, subtyping, and polymorphism out of untyped terms. Biorthogonality is also central to the study of classical and polarized calculi; see e.g. Curien and Munch-Maccagnoni [2010], Downen et al. [2020], and Munch-Maccagnoni [2009].

Most of the above uses are operational in character. The first use of biorthogonality in a denotational setting appears to be a paper by Abadi [2000], who uses it as a syntactic approximation to domain-theoretic admissibility. Katsumata [2008, 2005, 2013] showed that biorthogonality can be used to define logical relations over models of effects as a fibration.

Biorthogonality between denotational and operational semantics goes back to Benton and Hur [2009], who proved the correctness of a compiler from a call-by-value variant of PCF to a variant of Landin's SECD machine [1964]. This line of work has been extended to a polymorphic language with parametricity [Benton and Hur 2010], ML with mutable state and idealised assembly code [Hur and Dreyer 2011], and by others to various languages and machines; see e.g. the papers by Benton and Tabareau [2009] and Jaber and Tabareau [2010, 2011]. Of all these works, the one most similar to ours is the paper by Rodriguez et al. [2016], who prove the correctness of a compiler from

a call-by-name variant of PCF to the Krivine abstract machine, an environment machine similar to the CEK machine. Neither Benton and Hur [2009] nor Rodriguez et al. [2016] give the stacks of their machines a denotational interpretation, which complicates their logical relations.

Denotational Semantics of Stacks. The idea that the stacks have a denotational semantics appears to have been in the air at least since Reynolds's 1970s work on definitional interpreters [1998] and the pioneering work of Strachey and Wadsworth [1974, 2000] on continuations. To the best of our knowledge the first explicit account in the style used here is in the work of Streicher and Reus [1998]. The idea was then substantially developed by Paul Levy, and forms the basis of categorical models of CBPV [Levy 2005, 2003b]. Levy's adjunction models with stacks interpret values, computations, and stacks as the three kinds of morphism in a locally indexed adjunction. Bimodels retain this stack judgement, but make machine states and transitions primitive.

Logical Relations and CBPV. Logical relations for CBPV have mostly been studied denotationally. Kammar [2014] constructs relational models of an effect-annotated CBPV by lifting a pair of set-theoretic algebra models to a category of logical relations, with the relation on computations being the least one closed under returns and the algebraic operations. McDermott [2020] develops logical relations for graded CBPV using a suitable monad lifting. Azevedo de Amorim [2025] proves the soundness of an expected-cost semantics for a CBPV metalanguage via a logical relation built on a probabilistic coupling lifting. The only *operationally* defined biorthogonal logical relation for CBPV that we are aware of is that of New et al. [2019, 2021] on gradual type theory. Azevedo de Amorim et al. [2025] give a fibrational account of logical relations for categorical models of CBPV. They start from a locally indexed adjunction, and construct logical relations as fibrations over it, recovering biorthogonality in the style of Katsumata [2005] as a particular example [Azevedo de Amorim et al. 2025, §V.6]. Our models, by contrast, retain more intensional structure, making machine states and their transitions primitive. They can therefore prove intensional properties of abstract machines, such as adequacy and simulation between machines, that the fibrational approach does not currently address. The two approaches are nonetheless closely related. A syntactic bimodel does not satisfy the universal properties of a categorical model without substantial quotienting. Conversely, we conjecture that fibrational models with enough points should induce CK bimodels.

Goncharov et al. [2025] model both CBPV and fine-grained call-by-value as higher-order abstract GSOS specifications, and thereby derive applicative similarity, step-indexed logical relations, and soundness for contextual approximation *generically*, with each metatheorem reducing to a syntactic condition on the rules. Their framework abstracts over the *operational rules*, working with small-step reduction and Howe's method in place of abstract machines and biorthogonality.

Abstract Machines. A classic strand of work relates abstract machines to semantics by CPS transformation and defunctionalization: see Ager et al. [2003], Danvy [2005], and Reynolds [1998]. Another approach uses them as a basis for static analysis [Van Horn and Might 2010].

Acknowledgments

We thank Max S. New, Robert Atkey, and Pedro Azevedo de Amorim for helpful discussions. April Tune was supported by the UKRI Engineering and Physical Sciences Research Council (EPSRC) Doctoral Training Partnership, grant reference EP/W524414/1. Alex Kavvos was supported by a Royal Society Research Grant and a grant from the Advanced Research + Invention Agency (ARIA).

The accompanying Agda formalisation was developed by the authors. Large Language Models (LLMs) from the Anthropic Claude series were used to assist with expanding a small initial version into the full formalisation, and for substantial refactoring. All proofs were reviewed by the authors.

Data Availability Statement

The Agda formalisation accompanying this paper is available from Zenodo [Tune and Kavvos 2026]. It is available as source code and as a virtual-machine image with the required dependencies.

References

Martín Abadi. 2000. "TT-closed relations and admissibility." *Mathematical Structures in Computer Science*, 10, 3, 313–320. doi:10.1017/S0960129500003054.

Mads Sig Ager, Dariusz Biernacki, Olivier Danvy, and Jan Midtgaard. 2003. "A Functional Correspondence Between Evaluators and Abstract Machines." In: *Proceedings of the 5th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming*. Association for Computing Machinery, 8–19. doi:10.1145/888251.888254.

Thorsten Altenkirch and Ambrus Kaposi. 2016. "Type theory in type theory using quotient inductive types." In: *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. Association for Computing Machinery, 18–29. doi:10.1145/2837614.2837638.

Pedro H. Azevedo de Amorim, Satoshi Kura, and Philip Saville. 2025. "Logical relations for call-by-push-value models, via internal fibrations in a 2-category." In: *2025 40th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, Singapore, Singapore, 732–747. doi:10.1109/LICS65433.2025.00061.

Pedro Henrique Azevedo de Amorim. 2025. "Denotational Foundations for Expected Cost Analysis." *Proceedings of the ACM on Programming Languages*, 9, OOPSLA1, 280–306, 90. doi:10.1145/3720424.

Nick Benton and Chung-Kil Hur. 2009. "Biorthogonality, Step-Indexing and Compiler Correctness." In: *Proceedings of the ACM SIGPLAN International Conference on Functional Programming, ICFP*. ACM, 97–108. doi:10.1145/1596550.1596567.

Nick Benton and Chung-Kil Hur. 2010. *Realizability and Compositional Compiler Correctness for a Polymorphic Language*. Tech. rep. MSR-TR-2010-62. Microsoft Research. https://www.microsoft.com/en-us/research/publication/realizability-and-compositional-compiler-correctness-for-a-polymorphic-language/.

Nick Benton and Nicolas Tabareau. 2009. "Compiling functional types to relational specifications for low level imperative code." In: *TLDI '09: Proceedings of the 4th international workshop on Types in Language Design and Implementation*. Association for Computing Machinery, 3–14. doi:10.1145/1481861.1481864.

Pierre-Louis Curien. 1991. "An Abstract Framework for Environment Machines." *Theoretical Computer Science*, 82, 389–402. doi:10.1016/0304-3975(91)90230-Y.

Pierre-Louis Curien and Guillaume Munch-Maccagnoni. 2010. "The Duality of Computation under Focus." In: *Theoretical Computer Science*. Ed. by Cristian S. Calude and Vladimiro Sassone. Springer Berlin Heidelberg, Berlin, Heidelberg, 165–181. doi:10.1007/978-3-642-15240-5_13.

Olivier Danvy. 2005. "A Rational Deconstruction of Landin's SECD Machine." In: *Implementation and Application of Functional Languages*. Lecture Notes in Computer Science. Vol. 3474. Ed. by Clemens Grelck, Frank Huch, Greg J. Michaelson, and Phil Trinder. Springer Berlin Heidelberg, Berlin, Heidelberg, 52–71. doi:10.1007/11431664_4.

N. G. de Bruijn. 1972. "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem." *Indagationes Mathematicae*, 34, 5, 381–392. doi:10.1016/1385-7258(72)90034-0.

Stephan Diehl, Pieter Hartel, and Peter Sestoft. 2000. "Abstract machines for programming language implementation." *Future Generation Computer Systems*, 16, 7, 739–751. doi:10.1016/S0167-739X(99)00088-6.

Paul Downen, Philip Johnson-Freyd, and Zena M. Ariola. 2020. "Abstracting models of strong normalization for classical calculi." *Journal of Logical and Algebraic Methods in Programming*, 111, 100512. doi:10.1016/j.jlamp.2019.100512.

Matthias Felleisen and D.P. Friedman. 1987. "Control operators, the SECD-machine, and the λ-calculus." In: *Formal Description of Programming Concepts III*. Ed. by Martin Wirsing. North-Holland, 193–222. https://legacy.cs.indiana.edu/ftp/techreports/TR197.pdf.

Jean-Yves Girard. 1987. "Linear logic." *Theoretical Computer Science*, 50, 1, 1–101. doi:10.1016/0304-3975(87)90045-4.

Sergey Goncharov, Stelios Tsampas, and Henning Urbat. 2025. "Abstract Operational Methods for Call-by-Push-Value." *Proceedings of the ACM on Programming Languages*, 9, POPL, 35:1–35:27, 35. doi:10.1145/3704871.

Robert Harper. 2016. *Practical Foundations for Programming Languages*. (2nd ed.). Cambridge University Press. doi:10.1017/CBO9781316576892.

Chung-Kil Hur and Derek Dreyer. 2011. "A Kripke Logical Relation Between ML and Assembly." In: *Proceedings of the ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL*. ACM, 133–146. doi:10.1145/1926385.1926402.

Guilhem Jaber and Nicolas Tabareau. 2010. *Krivine realizability for compiler correctness*. Presented at Workshop LOLA 2010, Syntax and Semantics of Low Level Languages. (2010). https://hal.science/hal-00475210/.

Guilhem Jaber and Nicolas Tabareau. 2011. *The Journey of Biorthogonal Logical Relations to the Realm of Assembly Code*. Presented at LOLA 2011, Syntax and Semantics of Low-Level Languages. (2011). https://hal.science/hal-00594386/.

Yuchen Jiang, Runze Xue, and Max S. New. 2025. "Notions of Stack-Manipulating Computation and Relative Monads." *Proceedings of the ACM on Programming Languages*, 9, OOPSLA1, 563–589, 100. doi:10.1145/3720434.

Ohad Kammar. 2014. "An Algebraic Theory of Type-and-Effect Systems." Ph.D. Dissertation. University of Edinburgh. http://hdl.handle.net/1842/8910.

Shin-ya Katsumata. 2008. "A Characterisation of Lambda Definability with Sums Via ⊤⊤-Closure Operators." In: *Computer Science Logic* (Lecture Notes in Computer Science). Ed. by Michael Kaminski and Simone Martini. Vol. 5213. Springer Berlin Heidelberg, Berlin, Heidelberg, 278–292. doi:10.1007/978-3-540-87531-4_21.

Shin-ya Katsumata. 2005. "A Semantic Formulation of ⊤⊤-Lifting and Logical Predicates for Computational Metalanguage." In: *Computer Science Logic* (Lecture Notes in Computer Science). Ed. by Luke Ong. Vol. 3634. Springer Berlin Heidelberg, Berlin, Heidelberg, 87–102. doi:10.1007/11538363_8.

Shin-ya Katsumata. 2013. "Relating computational effects by ⊤⊤-lifting." *Information and Computation*, 222, 228–246. doi:10.1016/j.ic.2012.10.014.

G. A. Kavvos. 2025. "Adequacy for Algebraic Effects Revisited." *Proceedings of the ACM on Programming Languages*, 9, OOPSLA1, 927–955, 113. doi:10.1145/3720457.

Jean-Louis Krivine. 2009. "Realizability in classical logic." In: *Realizability in classical logic*. Vol. 27. Publisher: SMF. SMF, 197–229. https://hal.science/hal-00154500.

Jean-Louis Krivine. 2001. "Typed lambda-calculus in classical Zermelo-Frænkel set theory." *Archive for Mathematical Logic*, 40, 3, 189–205. doi:10.1007/s001530000057.

Peter J. Landin. 1964. "The Mechanical Evaluation of Expressions." *The Computer Journal*, 6, 4, 308–320. doi:10.1093/comjnl/6.4.308.

Xavier Leroy. 2009. "Formal verification of a realistic compiler." *Communications of the ACM*, 52, 7, 107–115. doi:10.1145/1538788.1538814.

Paul Blain Levy. 2003a. "Adjunction Models For Call-By-Push-Value With Stacks." *Electronic Notes in Theoretical Computer Science*, 69, 248–271. doi:10.1016/S1571-0661(04)80568-1.

Paul Blain Levy. 2005. "Adjunction Models For Call-By-Push-Value With Stacks." *Theory and Applications of Categories*, 14, 5, 75–110. doi:10.70930/tac/e17sf9pc.

Paul Blain Levy. 2003b. *Call-by-Push-Value: A Functional-Imperative Synthesis*. Semantic Structures in Computation. Springer Dordrecht. doi:10.1007/978-94-007-0954-6.

Paul Blain Levy. 1999. "Call-by-Push-Value: A Subsuming Paradigm." In: *Typed Lambda Calculi and Applications, TLCA* (Lecture Notes in Computer Science). Vol. 1581. Springer, 228–242. doi:10.1007/3-540-48959-2_17.

Paul Blain Levy. 2006. "Call-by-push-value: Decomposing call-by-value and call-by-name." *Higher-Order and Symbolic Computation*, 19, 4, 377–414. doi:10.1007/s10990-006-0480-6.

Sam Lindley and Ian Stark. 2005. "Reducibility and ⊤⊤-Lifting for Computation Types." In: *Typed Lambda Calculi and Applications* (Lecture Notes in Computer Science). Ed. by Paweł Urzyczyn. Vol. 3461. Springer Berlin Heidelberg, Berlin, Heidelberg, 262–277. doi:10.1007/11417170_20.

Ian Mason and Carolyn Talcott. 1991. "Equivalence in functional languages with effects." *Journal of Functional Programming*, 1, 3, 287–327. doi:10.1017/S0956796800000125.

Ian A. Mason, Scott F. Smith, and Carolyn L. Talcott. 1996. "From Operational Semantics to Domain Theory." *Information and Computation*, 128, 1, 26–47. doi:10.1006/inco.1996.0061.

Dylan McDermott. 2020. "Reasoning about Effectful Programs and Evaluation Order." Ph.D. Dissertation. University of Cambridge. doi:10.48456/tr-948.

P.-A. Mellies and J. Vouillon. 2005. "Recursive Polymorphic Types and Parametricity in an Operational Framework." In: *20th Annual IEEE Symposium on Logic in Computer Science (LICS' 05)*. IEEE, 82–91. doi:10.1109/LICS.2005.42.

Robin Milner. 1975. "Processes: A Mathematical Model of Computing Agents." In: *Logic Colloquium '73* (Studies in Logic and the Foundations of Mathematics). Vol. 80. Elsevier, 157–173. doi:10.1016/S0049-237X(08)71948-7.

Guillaume Munch-Maccagnoni. 2009. "Focalisation and Classical Realisability." In: *Computer Science Logic* (Lecture Notes in Computer Science). Ed. by Erich Grädel and Reinhard Kahle. Vol. 5771. Springer Berlin Heidelberg, Berlin, Heidelberg, 409–423. doi:10.1007/978-3-642-04027-6_30.

Max S. New, Daniel R. Licata, and Amal Ahmed. 2019. "Gradual type theory." *Proceedings of the ACM on Programming Languages*, 3, POPL, 15:1–15:31, 15. doi:10.1145/3290328.

Max S. New, Daniel R. Licata, and Amal Ahmed. 2021. "Gradual type theory." *Journal of Functional Programming*, 31, e21. doi:10.1017/S0956796821000125.

Simon L. Peyton Jones. 1987. *The Implementation of Functional Programming Languages*. Prentice Hall. https://www.microsoft.com/en-us/research/publication/the-implementation-of-functional-programming-languages-2/.

Benjamin C. Pierce. 2002. *Types and Programming Languages*. MIT Press. ISBN: 0-262-16209-1. https://www.cis.upenn.edu/~bcpierce/tapl/.

A. M. Pitts. 2011. "Howe's Method for Higher-Order Languages." In: *Advanced Topics in Bisimulation and Coinduction*. Cambridge Tracts in Theoretical Computer Science. Vol. 52. Ed. by D. Sangiorgi and J. Rutten. Cambridge University Press, 197–232. doi:10.1017/CBO9780511792588.006.

A. M. Pitts. 1997. "Operationally-Based Theories of Program Equivalence." In: *Semantics and Logics of Computation*. Publications of the Newton Institute. Ed. by A. M. Pitts and P. Dybjer. Cambridge University Press, 241–298. ISBN: 978-0-521-11846-0. doi:10.1017/CBO9780511526619.007.

A. M. Pitts. 2005. "Typed Operational Reasoning." In: *Advanced Topics in Types and Programming Languages*. Ed. by B. C. Pierce. The MIT Press, 245–289. doi:10.7551/mitpress/1104.003.0011.

Andrew Pitts and Ian Stark. 1998. "Operational Reasoning for Functions with Local State." In: *Higher Order Operational Techniques in Semantics*. Publications of the Newton Institute. Ed. by A. D. Gordon and A. M. Pitts. Cambridge University Press, 227–273. https://homepages.inf.ed.ac.uk/stark/operfl.pdf.

Andrew M. Pitts. 1998. "Existential types: Logical relations and operational equivalence." In: *Automata, Languages and Programming*. Vol. 1443. Ed. by Kim G. Larsen, Sven Skyum, and Glynn Winskel. Springer Berlin Heidelberg, Berlin, Heidelberg, 309–326. ISBN: 978-3-540-64781-2. doi:10.1007/BFb0055063.

Andrew M. Pitts. 2000. "Parametric polymorphism and operational equivalence." *Mathematical Structures in Computer Science*, 10, 3, 321–359. doi:10.1017/S0960129500003066.

Andrew M. Pitts. 2010. "Step-Indexed Biorthogonality: a Tutorial Example." In: *Modelling, Controlling and Reasoning About State* (Dagstuhl Seminar Proceedings (DagSemProc)). Ed. by Amal Ahmed, Nick Benton, Lars Birkedal, and Martin Hofmann. Vol. 10351. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 1–10. doi:10.4230/DagSemProc.10351.6.

G. D. Plotkin. 1973. *Lambda Definability and Logical Relations*. Memorandum SAI-RM-4. University of Edinburgh. https://homepages.inf.ed.ac.uk/gdp/publications/logical_relations_1973.pdf.

Gordon D Plotkin. 2004a. "The origins of structural operational semantics." *The Journal of Logic and Algebraic Programming*, 60–61, 3–15. doi:10.1016/j.jlap.2004.03.009.

Gordon D. Plotkin. 2004b. "A structural approach to operational semantics." *The Journal of Logic and Algebraic Programming*, 60–61, 17–139. doi:10.1016/j.jlap.2004.05.001.

Gordon D. Plotkin. 1977. "LCF considered as a programming language." *Theoretical Computer Science*, 5, 3, 223–255. doi:10.1016/0304-3975(77)90044-5.

John C. Reynolds. 1998. "Definitional Interpreters for Higher-Order Programming Languages." *Higher-Order and Symbolic Computation*, 11, 4, 363–397. doi:10.1023/A:1010027404223.

Leonardo Rodriguez, Miguel Pagano, and Daniel Fridlender. 2016. "Proving Correctness of a Compiler Using Step-indexed Logical Relations." *Electronic Notes in Theoretical Computer Science*, 323, 197–214. doi:10.1016/j.entcs.2016.06.013.

Ian Stark. 1996. "A Fully Abstract Domain Model for the π-Calculus." In: *Proceedings of the 11th Annual IEEE Symposium on Logic in Computer Science (LICS 1996)*. IEEE Computer Society, 36–42. doi:10.1109/LICS.1996.561301.

Christopher Strachey and Christopher P. Wadsworth. Jan. 1974. *Continuations: A Mathematical Semantics for Handling Full Jumps*. Technical Monograph PRG-11. Oxford University Computing Laboratory, Programming Research Group, Oxford, (Jan. 1974). https://www.cs.ox.ac.uk/files/3233/PRG11.pdf.

Christopher Strachey and Christopher P. Wadsworth. 2000. "Continuations: A Mathematical Semantics for Handling Full Jumps." *Higher-Order and Symbolic Computation*, 13, 1, 135–152. doi:10.1023/A:1010026413531.

Th. Streicher and B. Reus. 1998. "Classical logic, continuation semantics and abstract machines." *Journal of Functional Programming*, 8, 6, 543–572. doi:10.1017/S0956796898003141.

[SW] April Tune and Alex Kavvos, *Bimodels and Biorthogonality for Abstract Machines* June 2026. doi:10.5281/zenodo.21490016, URL: https://doi.org/10.5281/zenodo.21490016.

David Van Horn and Matthew Might. 2010. "Abstracting Abstract Machines." In: *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (ICFP '10). Association for Computing Machinery, 51–62. doi:10.1145/1863543.1863553.

Jerome Vouillon and Paul-André Melliès. 2004. "Semantic types: a fresh look at the ideal model for types." In: *Proceedings of the 31st ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. Association for Computing Machinery, 52–63. doi:10.1145/964001.964006.

A.K. Wright and M. Felleisen. 1994. "A Syntactic Approach to Type Soundness." *Information and Computation*, 115, 1, 38–94. doi:10.1006/inco.1994.1093.

Received 2026-03-03; accepted 2026-05-13